Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 10

Prof. Claudio Benossi

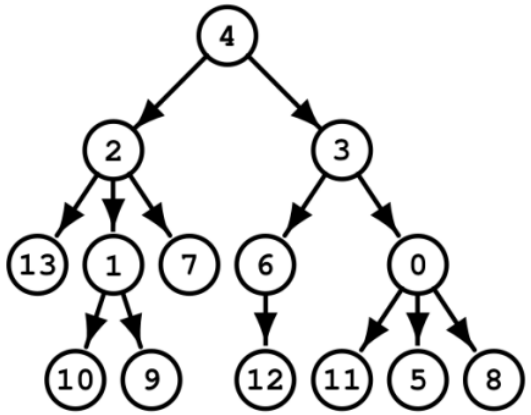
1. Unidade

Árvores

Árvore: um tipo especial de grafo

Árvores são **grafos acíclicos**.

- São estruturas de dados essencialmente dinâmicas, não-lineares, que permite a inserção, pesquisa e destruição de elementos.
- É composta por um conjunto de nós. Existe um nó denominado raiz que contém zero ou mais subárvores e cada sub-árvore pode ser considerada uma árvore.
- Com exceção do elemento do topo, cada elemento da árvore possui um elemento **pai** e zero ou mais elementos **filhos**.



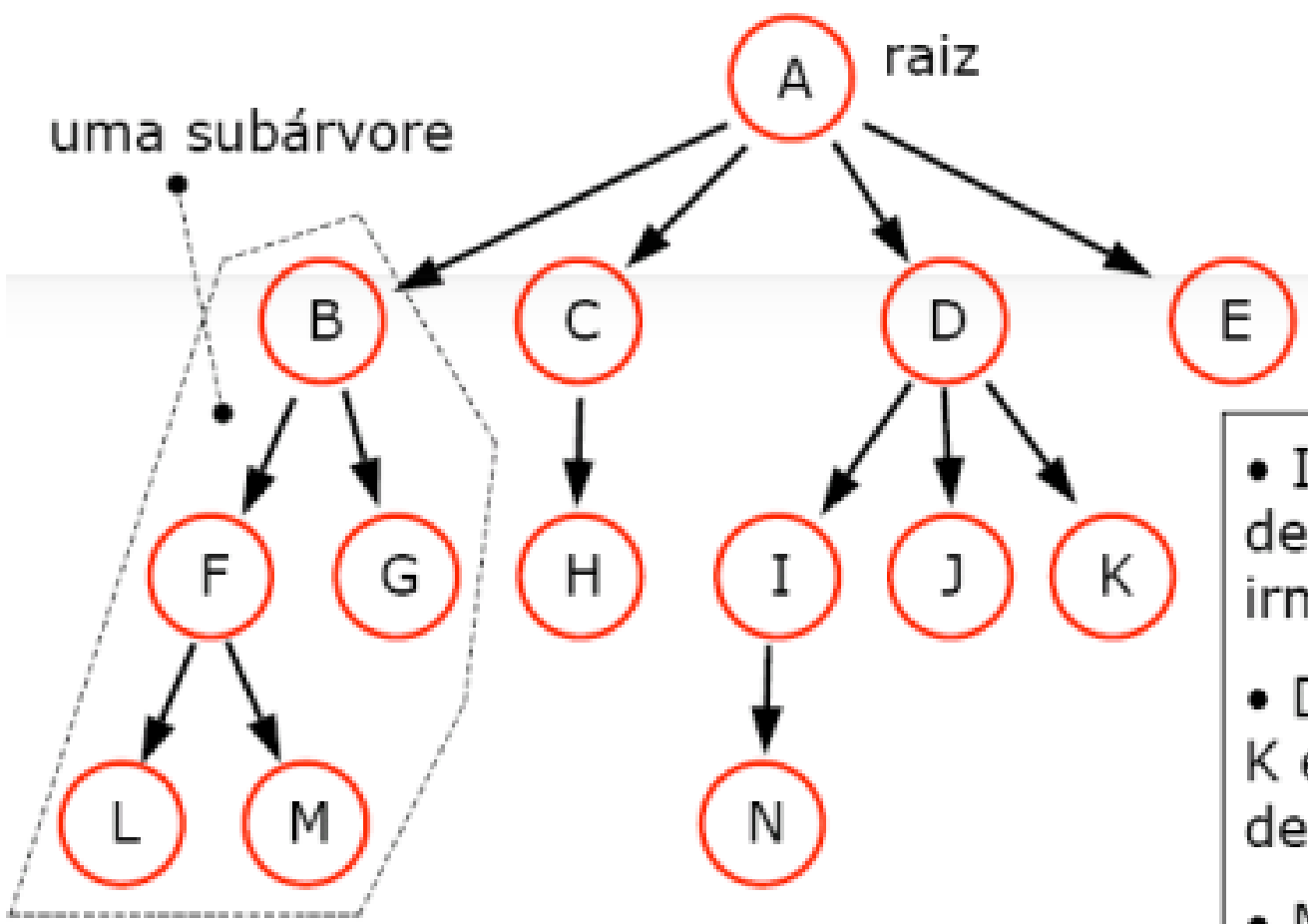
Árvore: Definição formal

Formalmente, define-se uma árvore T como um conjunto de nós que armazenam elementos em relacionamento pai-filho com as seguintes propriedades:

Se T não é vazia, ela tem um nó especial chamado de raiz de T que não tem pai.

Cada nó v de T diferente da raiz tem um único nó pai, w. Todo nó com pai w é filho de w.

Árvore: Propriedades



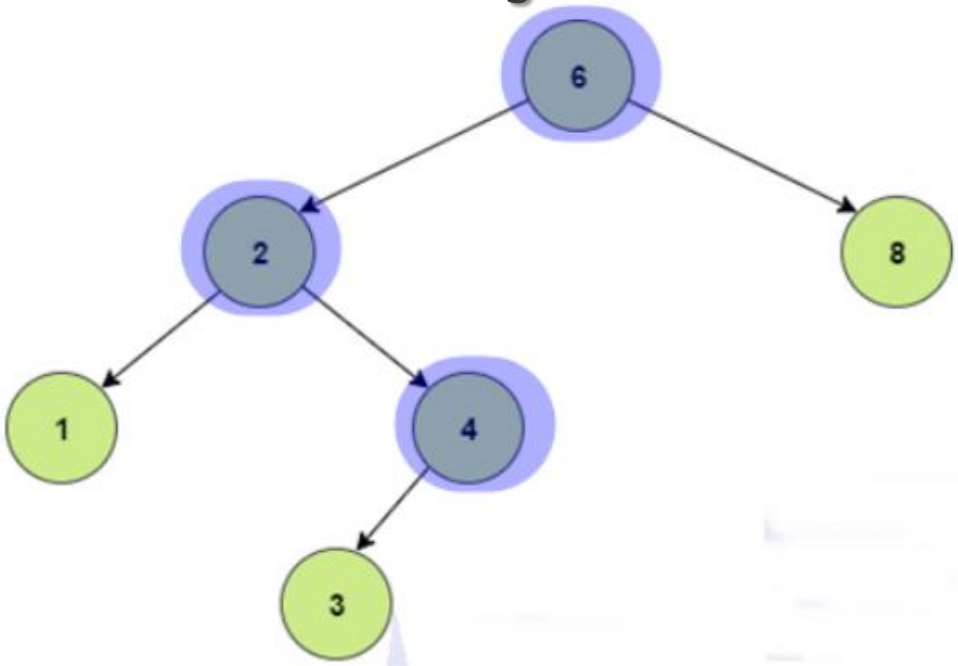
- I, J, K são filhos de D, logo são irmãos
- D é pai de I, J, K e antepassado de N
- N é descendente de D

- nós internos: A, B, C, D, F, I
- nós externos ou folhas: E, G, H, J, K, L, M

Árvore: Propriedades

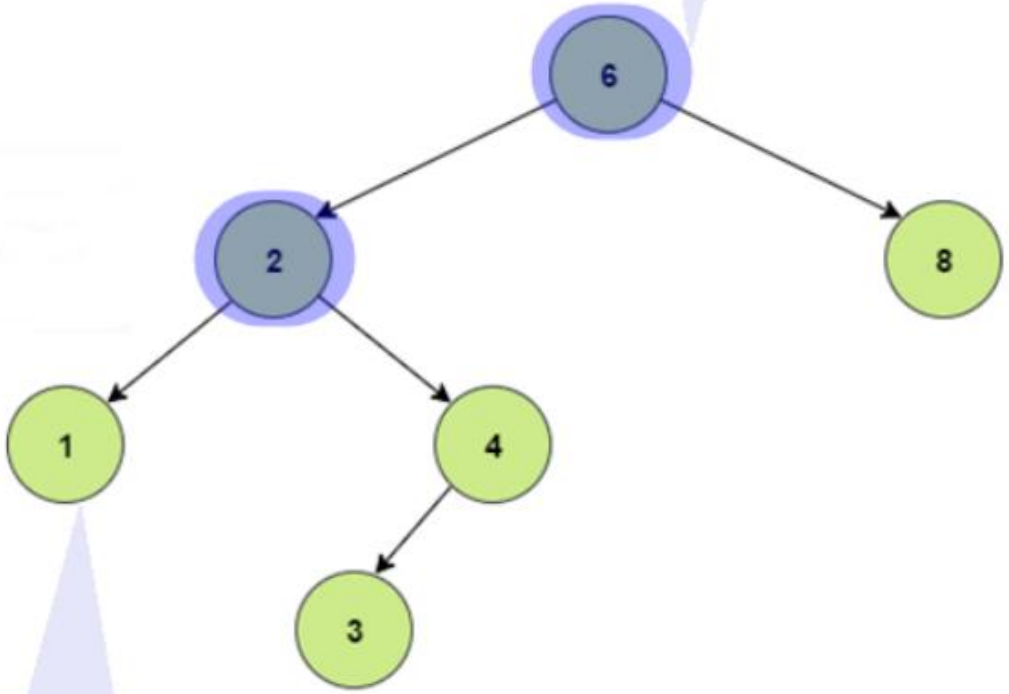
- Árvore vazia significa que a árvore T não tem nós.
- Dois nós que são filhos do mesmo pai são irmãos.
- Um nó v é externo se v não tem filhos.
- Um nó v é interno se tem um ou mais filhos.
- Nós externos também são conhecidos como folhas.
- Um nó u é ancestral de um nó v, se u=v, ou u é ancestral do pai de v.
- Um nó v é descendente de um nó u se u é ancestral de v.
- A subárvore de T enraizada no nó v é a árvore que consiste em todos os descendentes de v em T (incluindo o próprio v).

Árvore: Definição formal



Os nós 2, 4 e 6 são ancestrais do no 3

O nó 6 não possui ancestrais



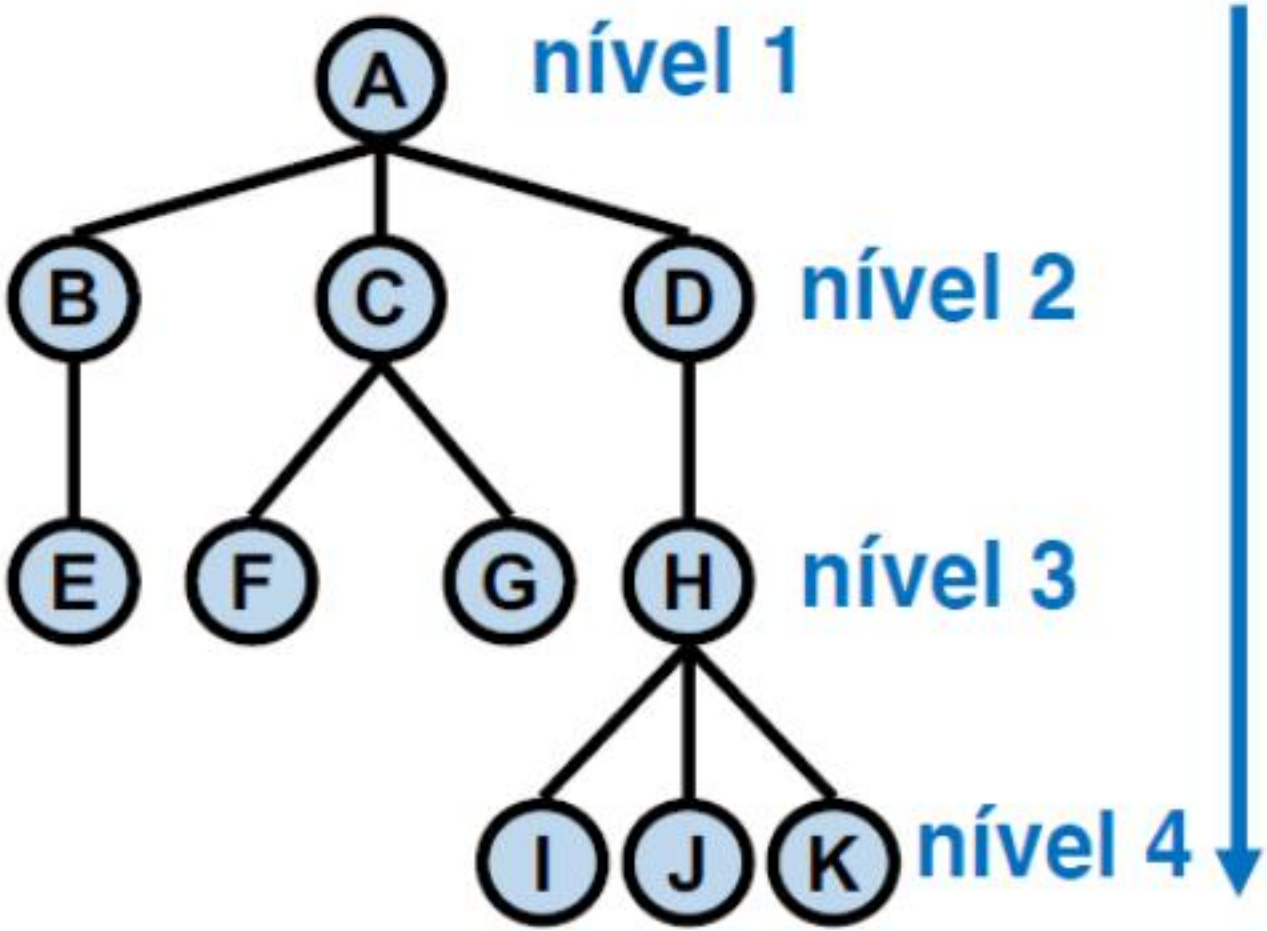
Os nós 2 e 6 são ancestrais do nó 1

Árvore: Definição formal

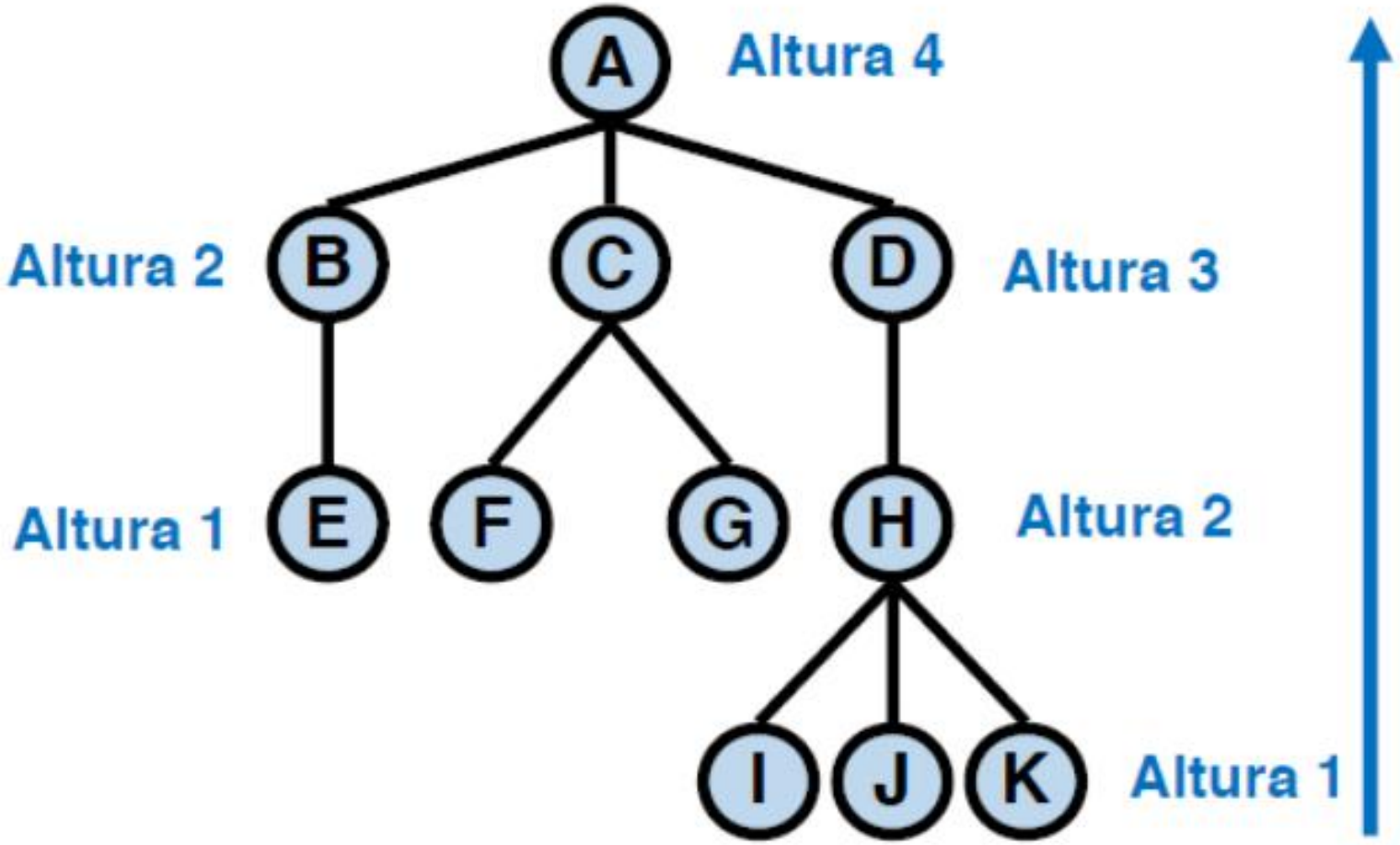
A convenção de árvore vazia permite que definir uma árvore recursivamente, de maneira que uma árvore T ou está vazia ou consiste em um nó r, chamado de raiz de T, e um conjunto (possivelmente vazio) de árvores cujas raízes são filhas de r.

Na maioria dos S.O., os arquivos são organizados hierarquicamente em diretórios aninhados (também chamados de pastas) que são apresentados aos usuários sob a forma de uma árvore.

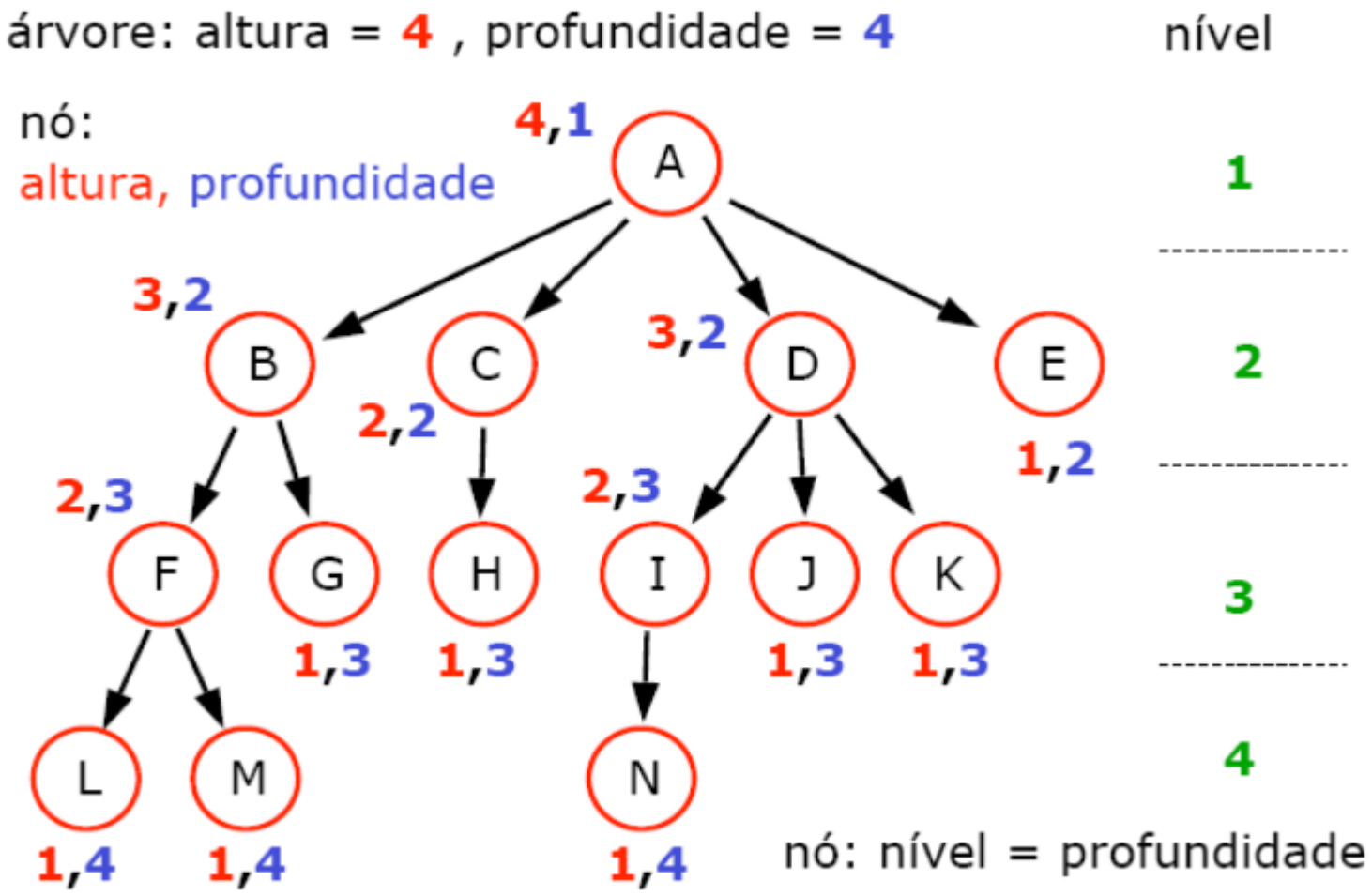
Nível da Árvore



Altura da Árvore



Árvore - Nível de Profundidade

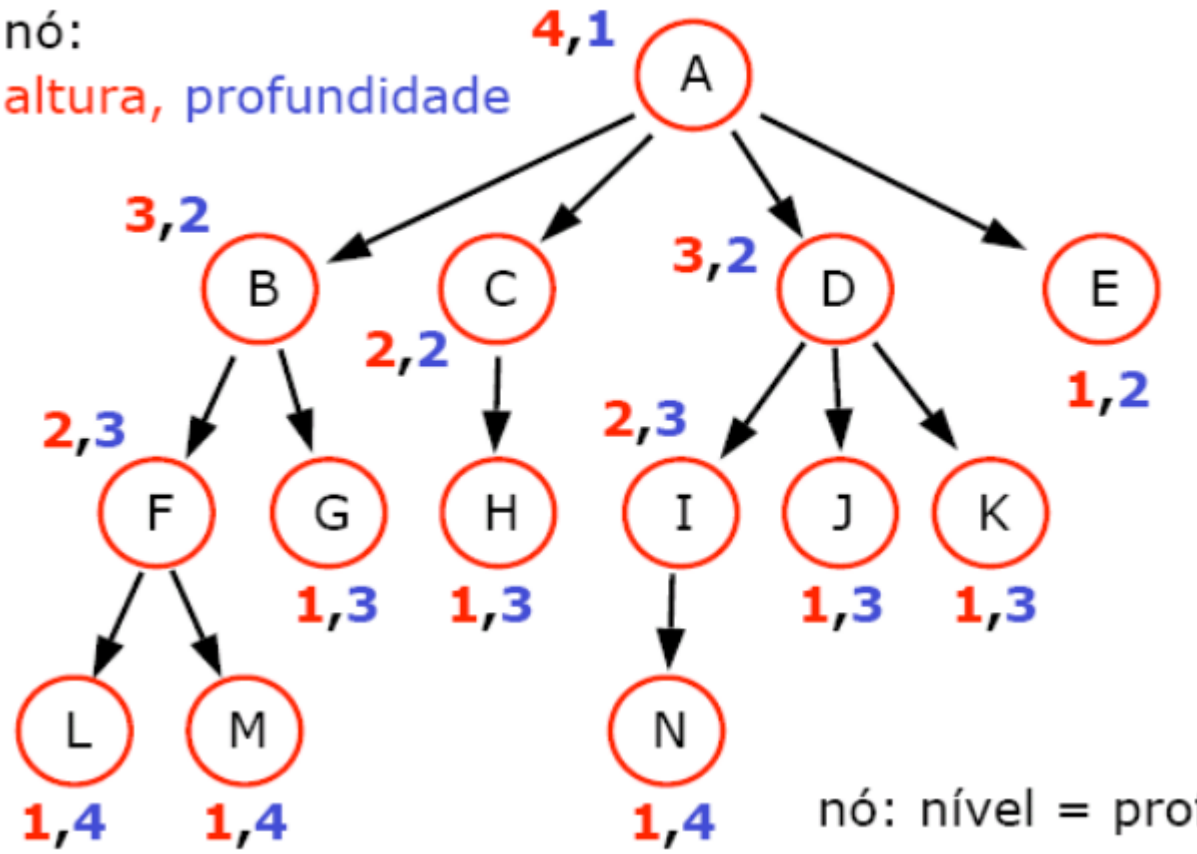


- **Raiz** - é o nó de mais alto nível da árvore (primeiro nó da árvore).
- **Nível** - o número de nós que vai da raiz até um determinado nó.
- **Grau** - o número de subárvores de um nó
- **Folha** - nó com grau igual a zero
- **Altura** - nível mais alto da árvore
- **Pai** - a raiz de uma subárvore é pai das raízes de suas subárvores
- **Filho** - os nós raízes de uma subarvores é filho da raiz da árvore
- **Irmão** - nós raízes das subárvores são irmãos

Árvore - Nível de Profundidade

árvore: altura = 4 , profundidade = 4

nó:
altura, profundidade



nível

1

2

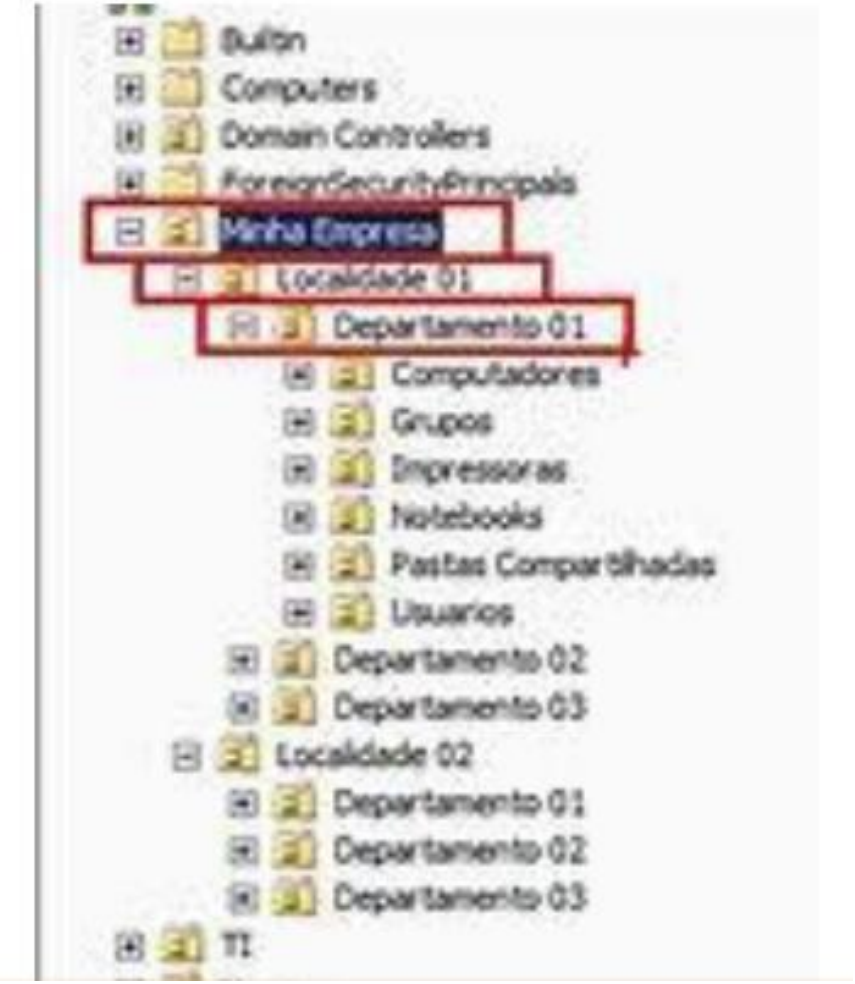
3

4

nó: nível = profundidade

Exemplos de estruturas em forma de árvores

- O organograma de uma empresa;
- A divisão de um livro em capítulos, seções, tópicos, etc;
- A árvore genealógica de uma pessoa.
- Na maioria dos S.O., os arquivos são organizados hierarquicamente em diretórios aninhados (também chamados de pastas) que são apresentados aos usuários sob a forma de uma árvore.



Árvores: Definição

Uma árvore é um conjunto finito de um ou mais nós (ou vértices) tal que:

- existe um nó denominado **raiz**;
- os demais nós formam m>= 0 conjuntos disjuntos s_1 , s_2 , ... , s_m , tais que cada um desses conjuntos também é uma árvore (denominado sub-árvore).

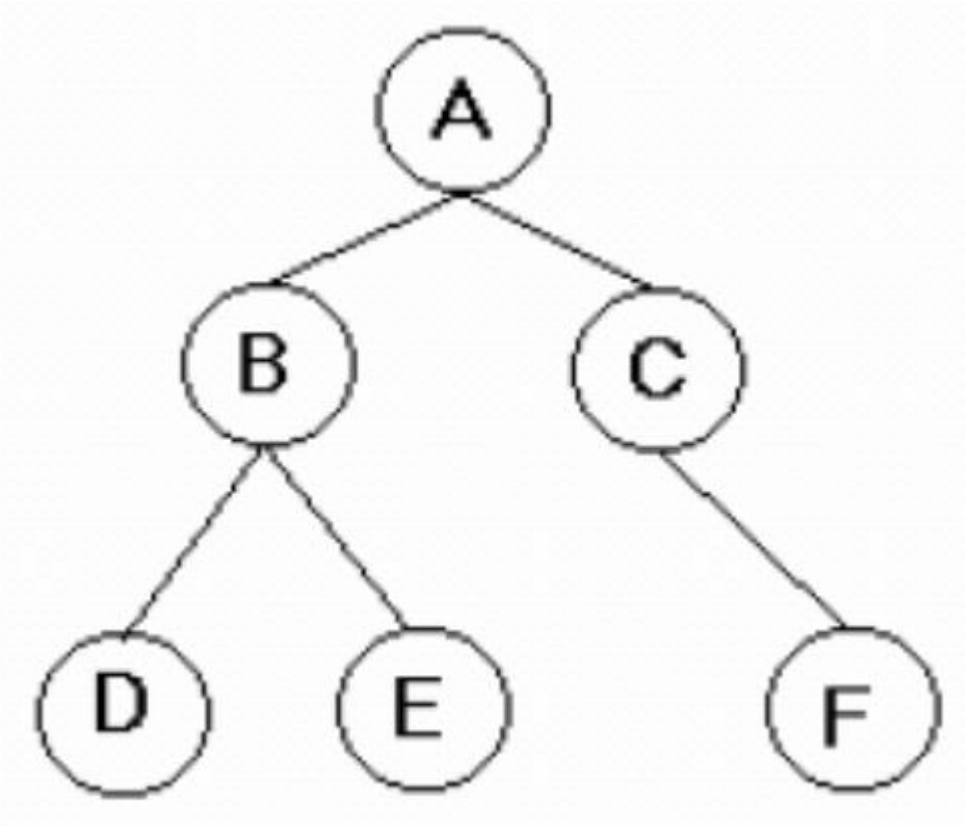
Floresta

Uma floresta é um conjunto de árvores.

- Se v é um nó de T, a notação Tv indica a subárvore de T com raiz v.
- Há formas diferentes de representações gráficas de uma árvore. Em todas elas, cada nó poderá ser associado a um identificador, denominado rótulo.

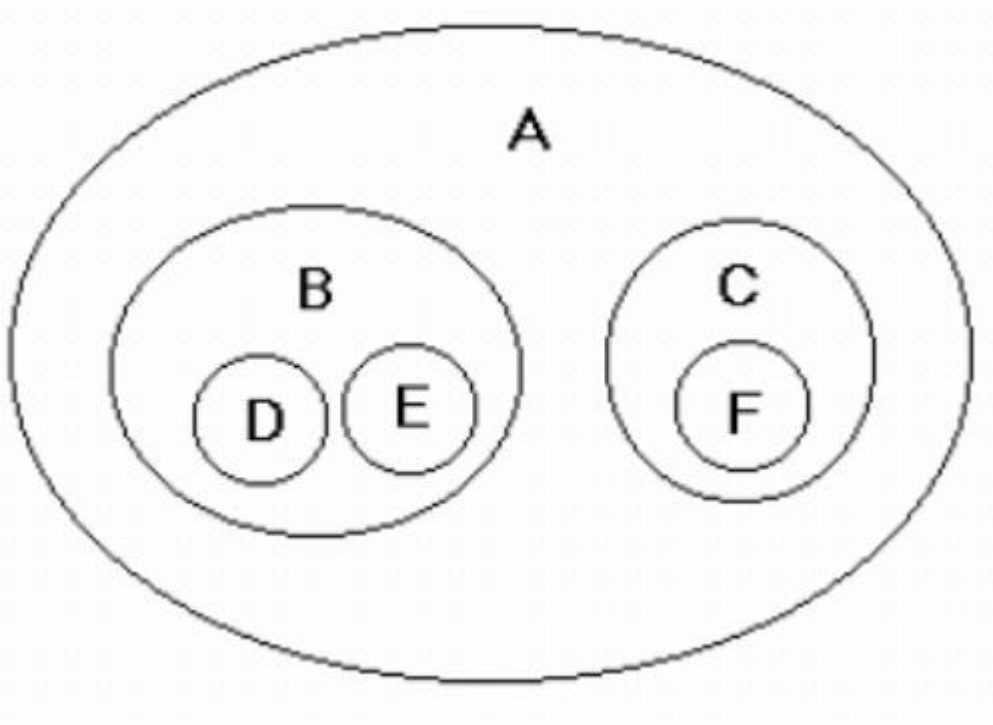
Árvores: representações básicas

a) Representação hierárquica



b) Representação por conjuntos

(diagrama de inclusão)



Árvores: representações básicas

c) Representação por expressão parentetizada (parênteses aninhados)

- Cada conjunto de parênteses correspondentes contém um nó e seus filhos. Se um nó não tem filhos, ele é seguido por um par de parênteses sem conteúdo.

(A(B(D()E())))(C(F()))

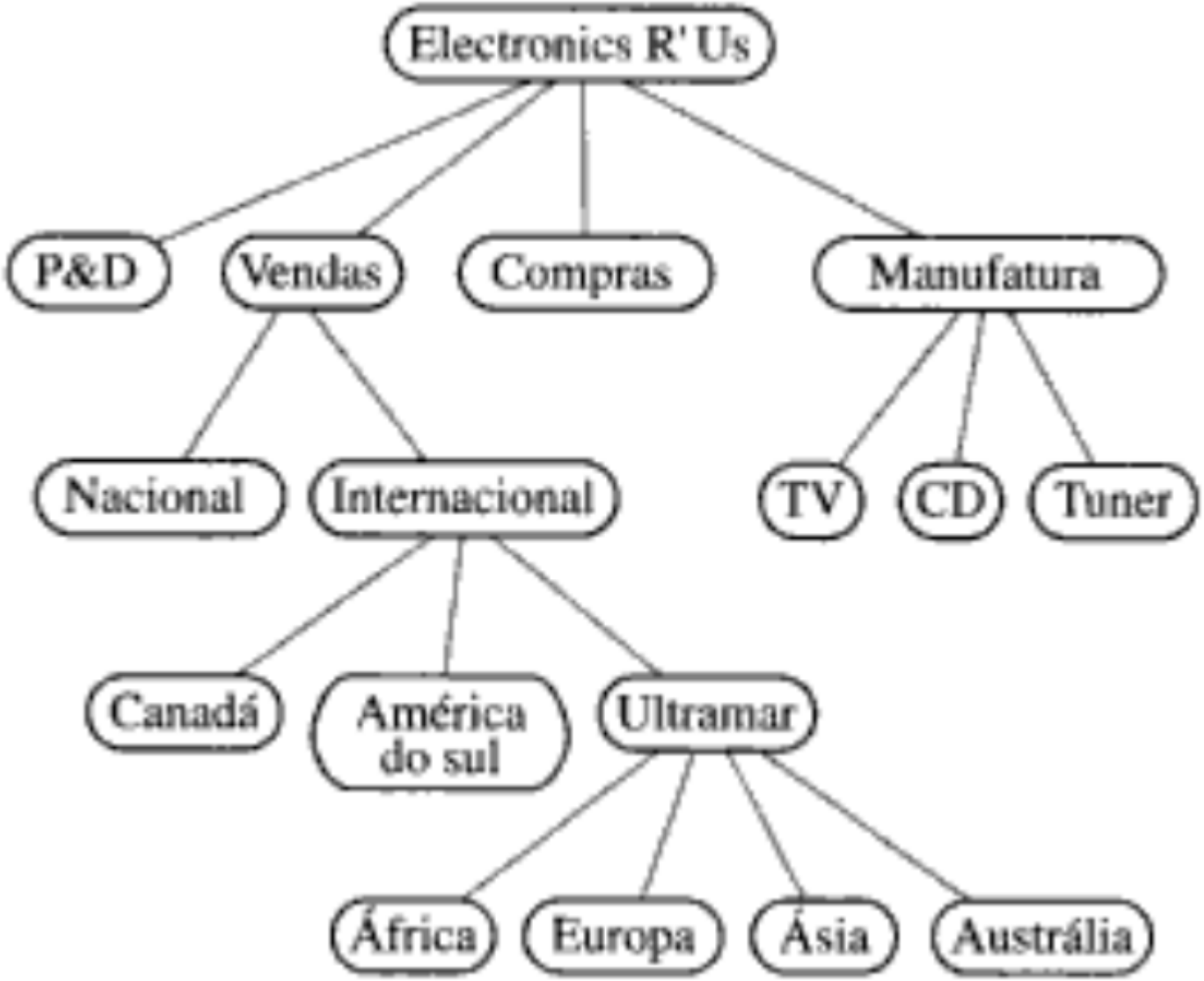
Árvores: representações básicas

d) Representação por expressão não-parentetizada

- Cada nó é seguido por um número que indica a quantidade de filhos desse nó, e em seguida por esses filhos, representados do mesmo modo.

A 2 B 2 D 0 E 0 C 1 F 0

Exemplo de Representação de Árvore



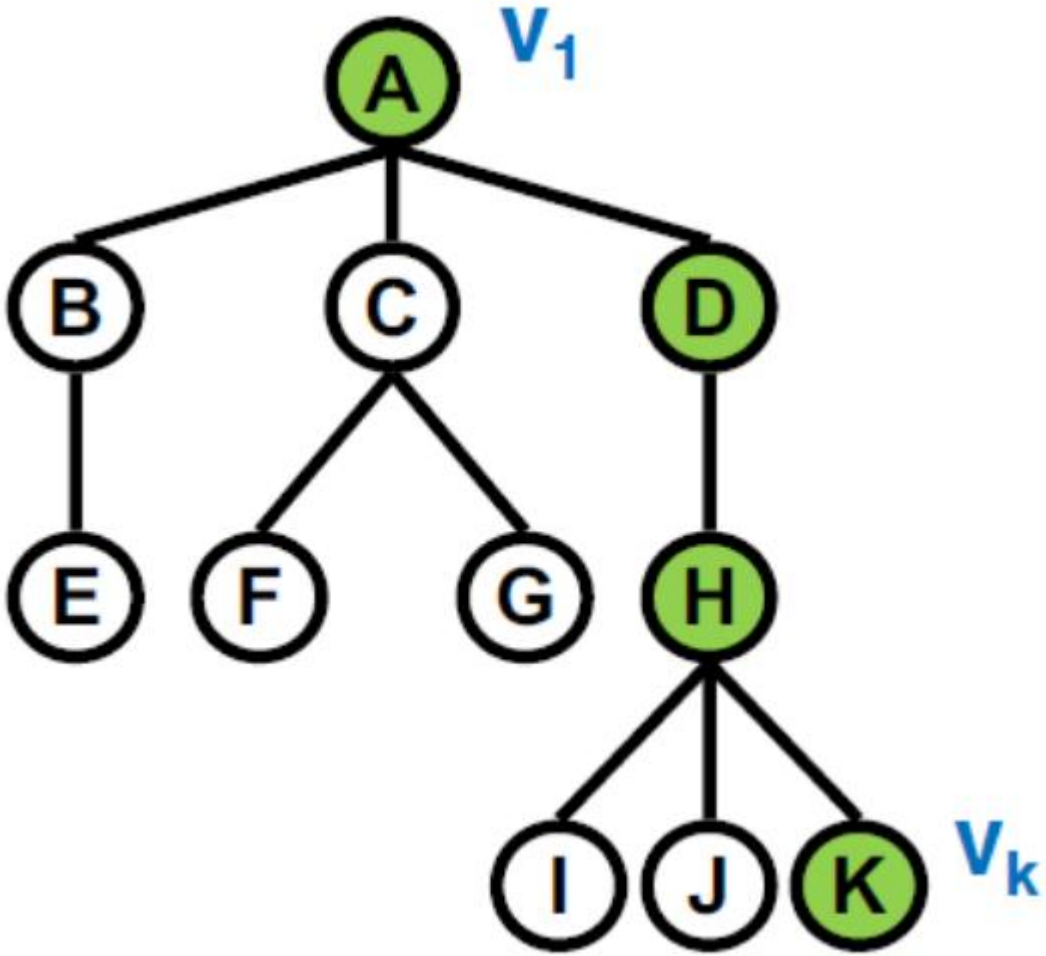
Raiz: o elemento do topo (desenhado como o elemento mais alto, com todos os demais conectados abaixo). Com exceção do elemento do topo (a raiz), cada elemento da árvore possui um nó **pai** e zero ou mais nós **filhos**.

Caminhos em Árvores

Dada uma árvore qualquer:

- Existe caminho entre dois nós u e v da árvore, se a partir do nó u puder chegar ao nó v percorrendo-se as arestas que ligam os nós intermediários entre u e v.
- Existe sempre um caminho entre a raiz e qualquer nó da árvore.
- Um caminho de T é uma sequência de nós tais que quaisquer dois nós consecutivos da sequência formam uma aresta.
- Diz-se que existe caminho entre dois nodos V e W da árvore, se a partir do nodo V puder chegar ao nodo W percorrendo-se as arestas que ligam os nodos intermediários entre V e W.

Caminho em Árvore

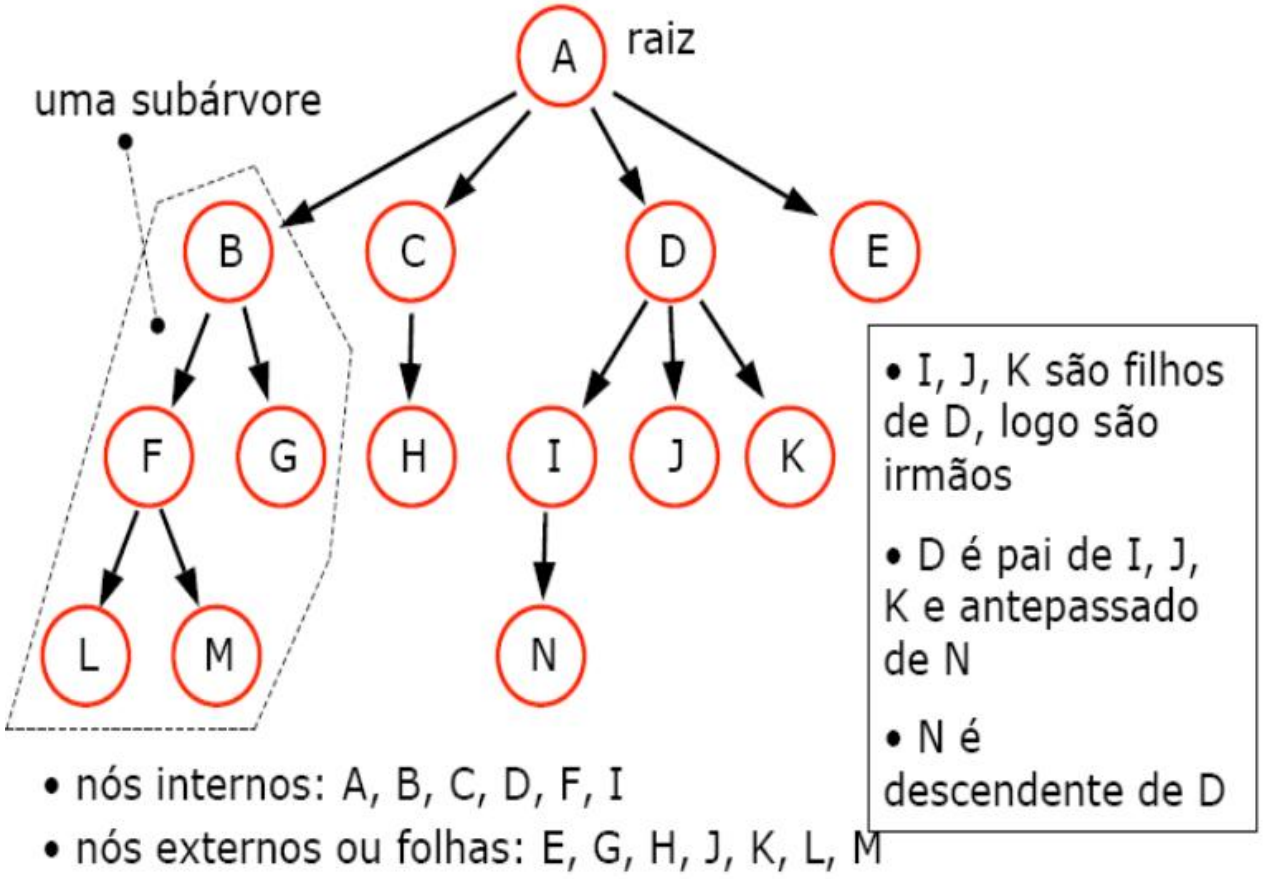


Um caminho é uma sequência de nós consecutivos distintos entre dois nós.

Caminhos em Árvores

Dada uma árvore qualquer:

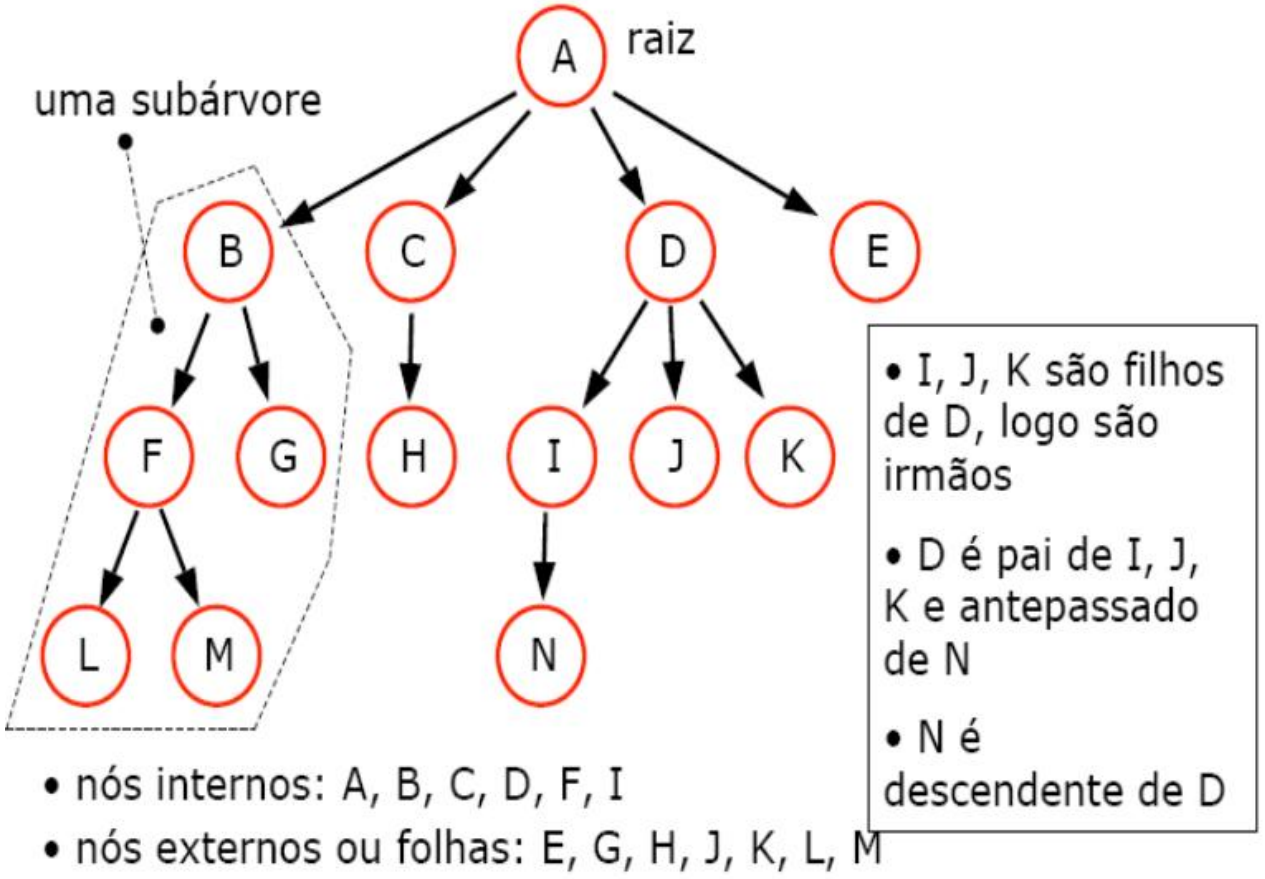
- Se houver um caminho entre u e v, começando em u diz-se que u é um nó ancestral de v e v é um nó descendente de u.
- Se este caminho contiver uma única aresta, diz-se que u é o nó pai de v e que v é um nó filho de u.



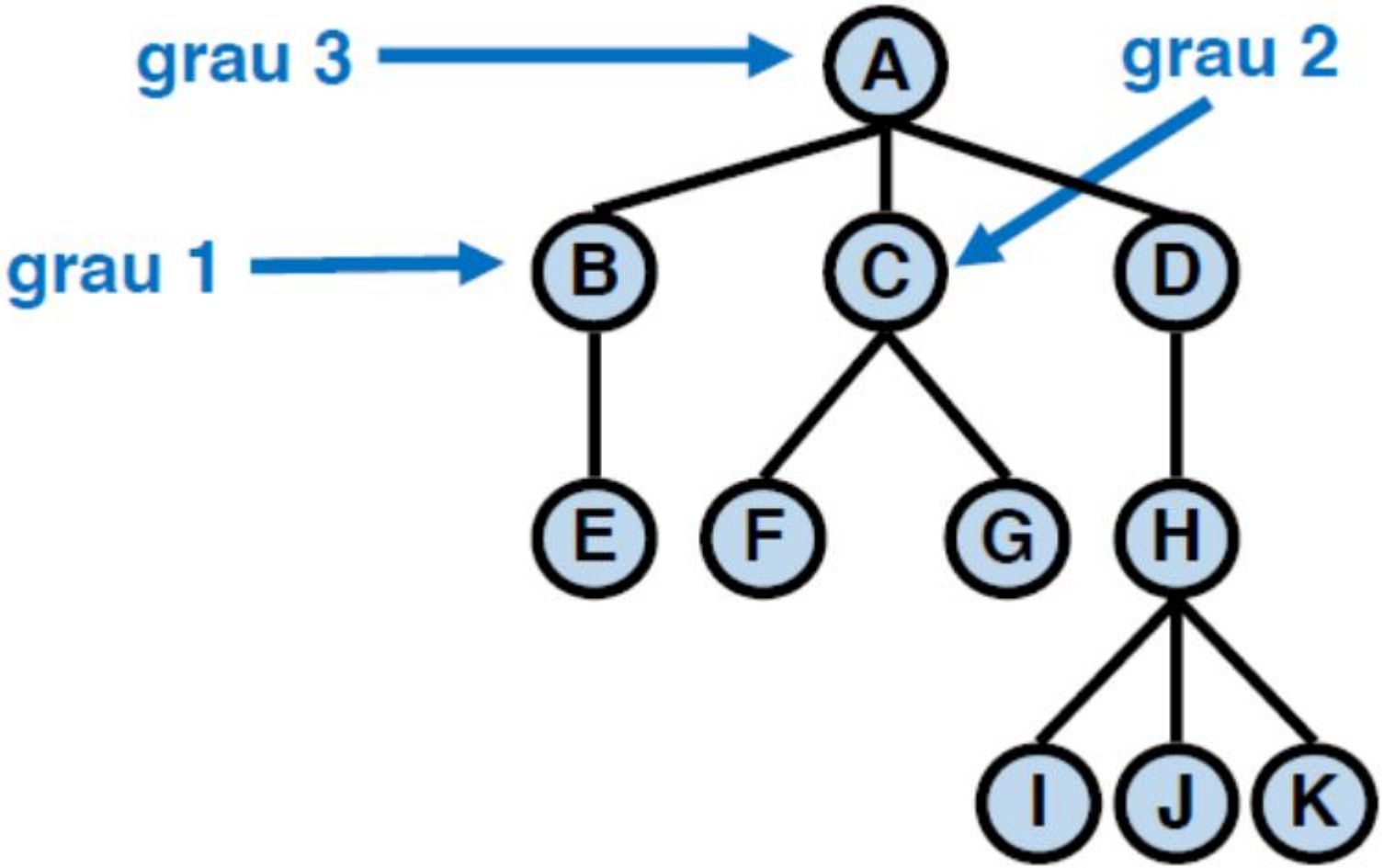
Caminhos em Árvores

Dada uma árvore qualquer:

- Dois nós que são nós filhos do mesmo nó pai são denominados nós irmãos.
- Uma característica inerente a árvores é que qualquer nó, exceto a raiz, tem um único nó pai.
- Se um nó não possui nós descendentes, ele é chamado de folha ou nó terminal da árvore.



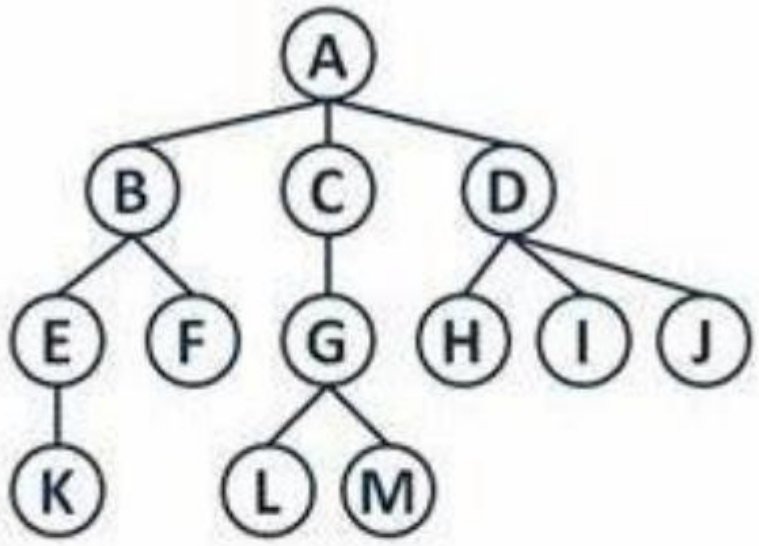
Grau de um nó



Grau de um nó

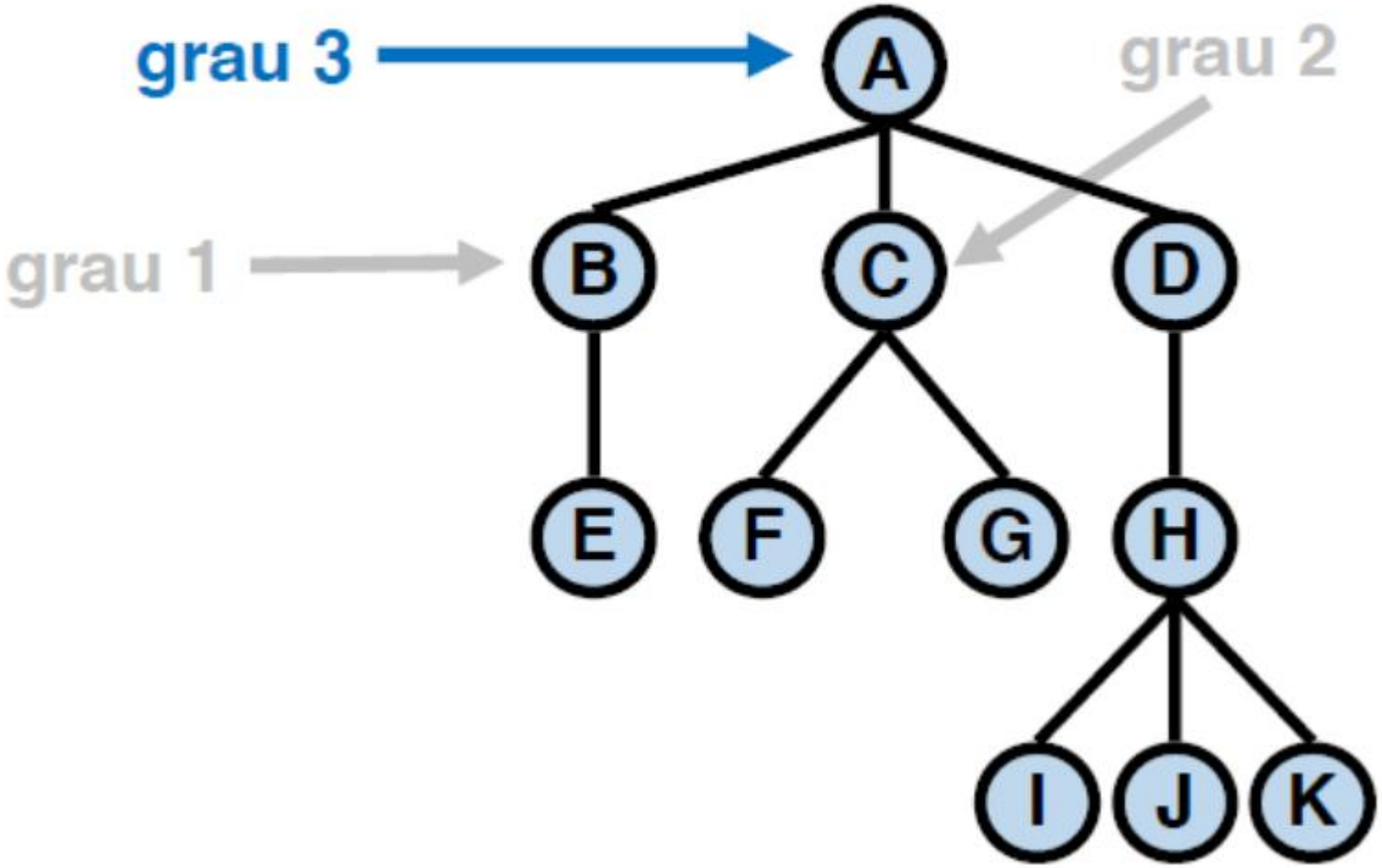
Dada uma árvore qualquer:

- Grau de um nó é igual ao número de filhos, ou o grau de saída.
- Um nó folha tem grau zero.
- O grau da árvore é igual ao grau do nó de maior grau da árvore.

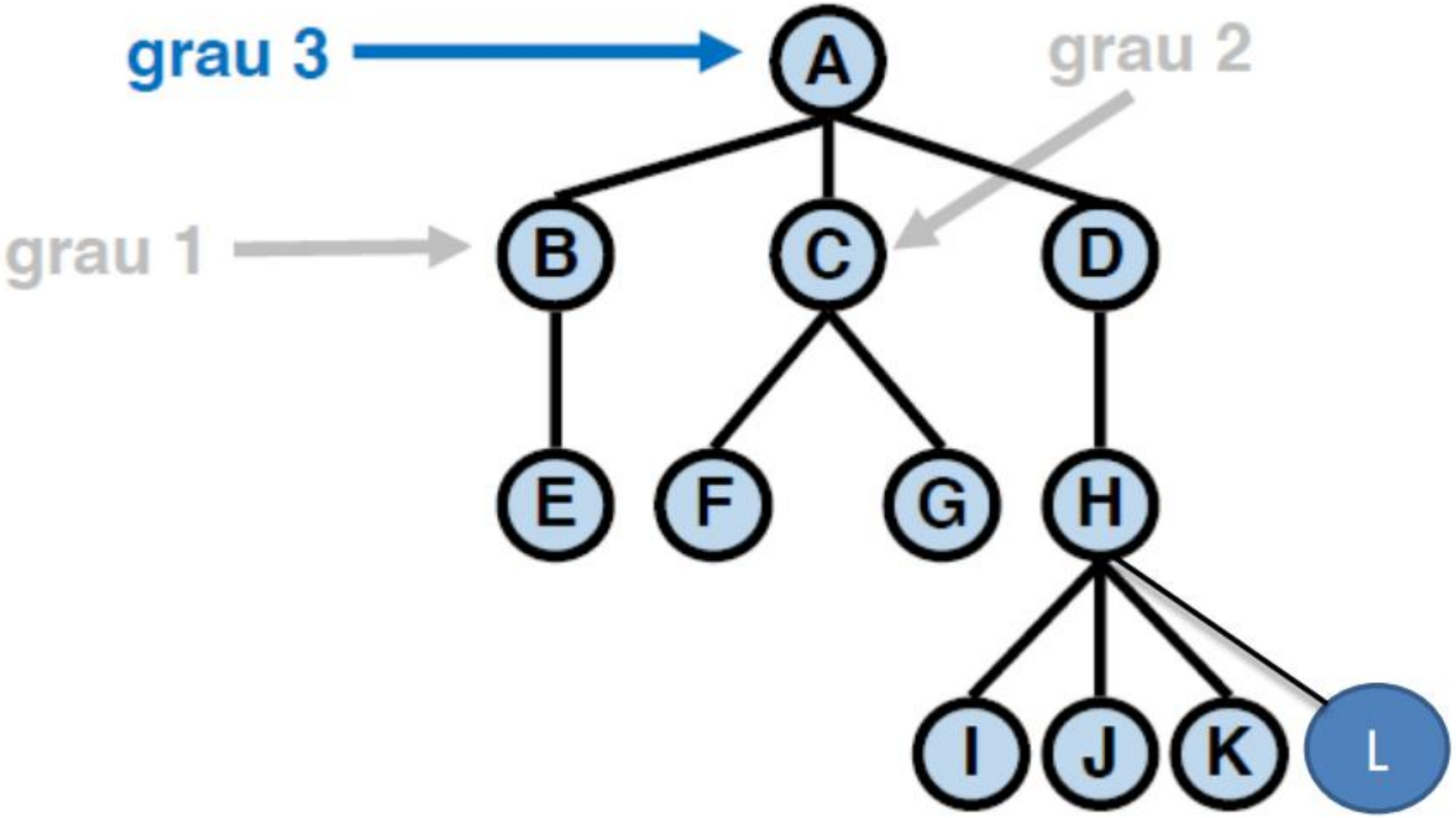


Nó	Grau
A, D	3
B, G	2
C, E	1
K, F, L, M, H, I, J	0

Grau da Árvore



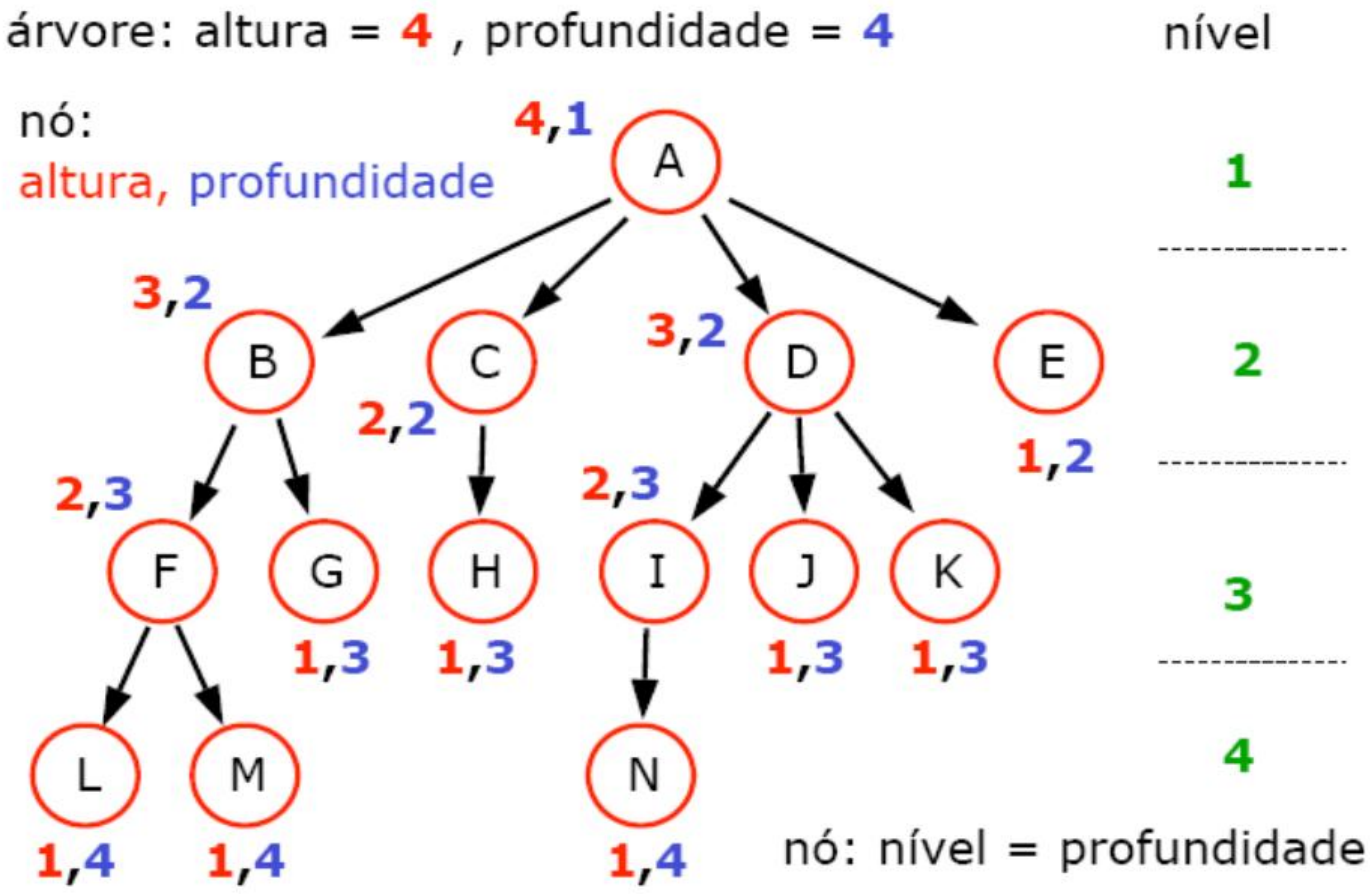
Grau da Árvore é 4



Nível de Profundidade da Árvore

O nível da árvore é igual ao nível do nó de maior nível da árvore.

O Nível de um nó é o número de nós existentes no caminho entre a raiz e o próprio nó. A raiz tem nível 1.



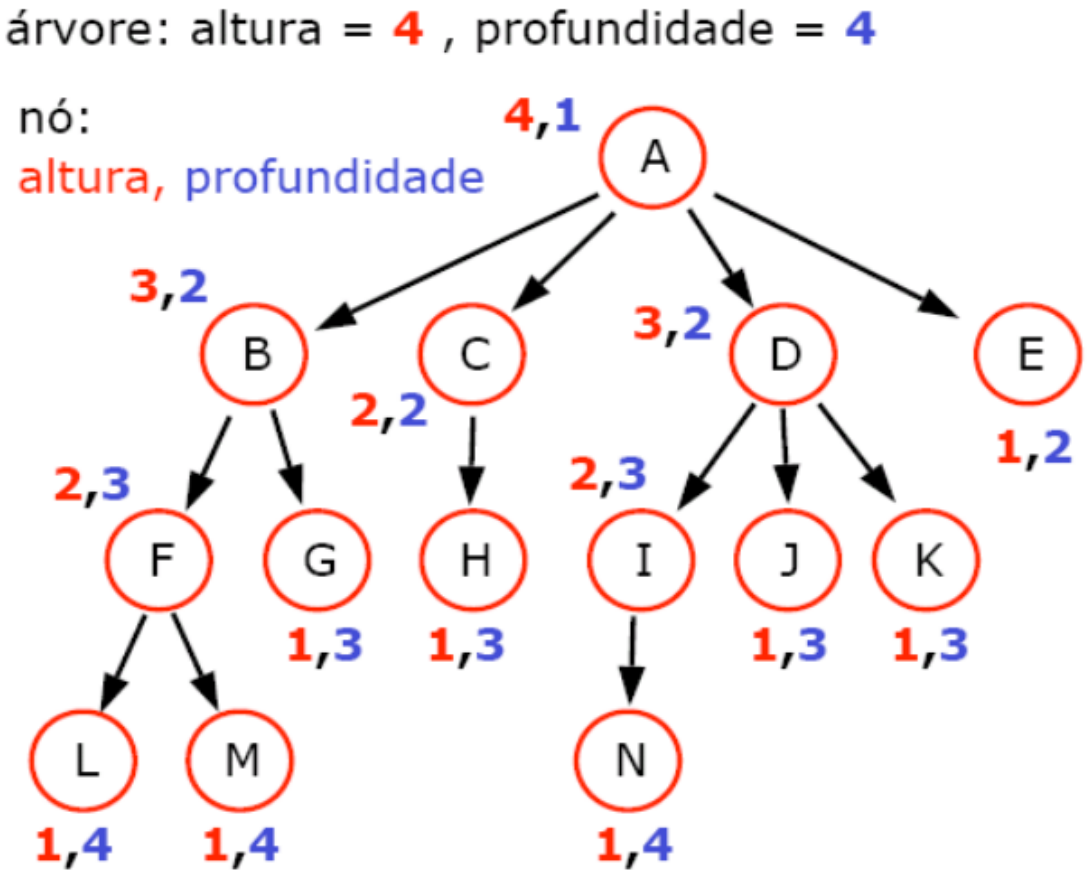
Altura da Árvore

Altura:

- do **nó**: comprimento dos maiores caminhos do nó a uma folha;
- da **árvore**: a altura da raiz.

Convenciona-se que:

- A altura de uma folha é 1.
- A altura da árvore vazia é 0.



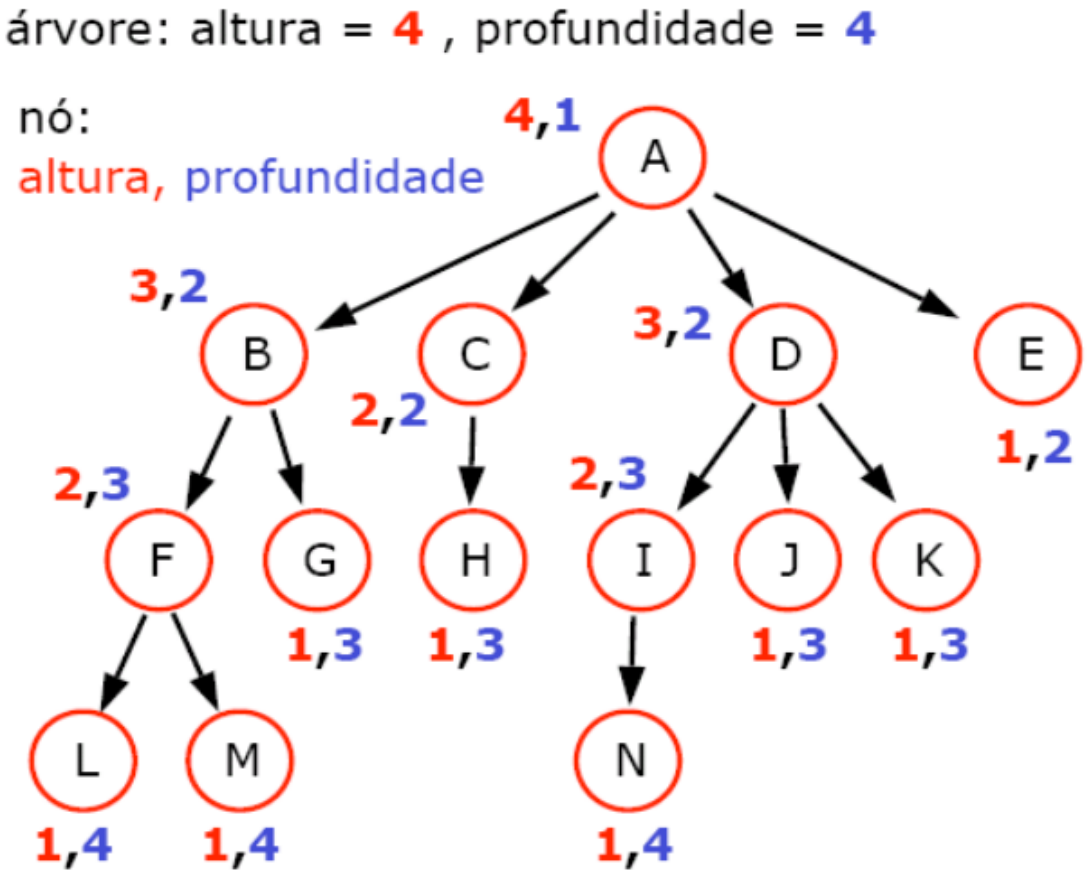
Altura e Profundidade da Árvore

Profundidade

- do nó: comprimento do (único) caminho da raiz ao nó.
- da árvore: máximo das profundidades das suas folhas.

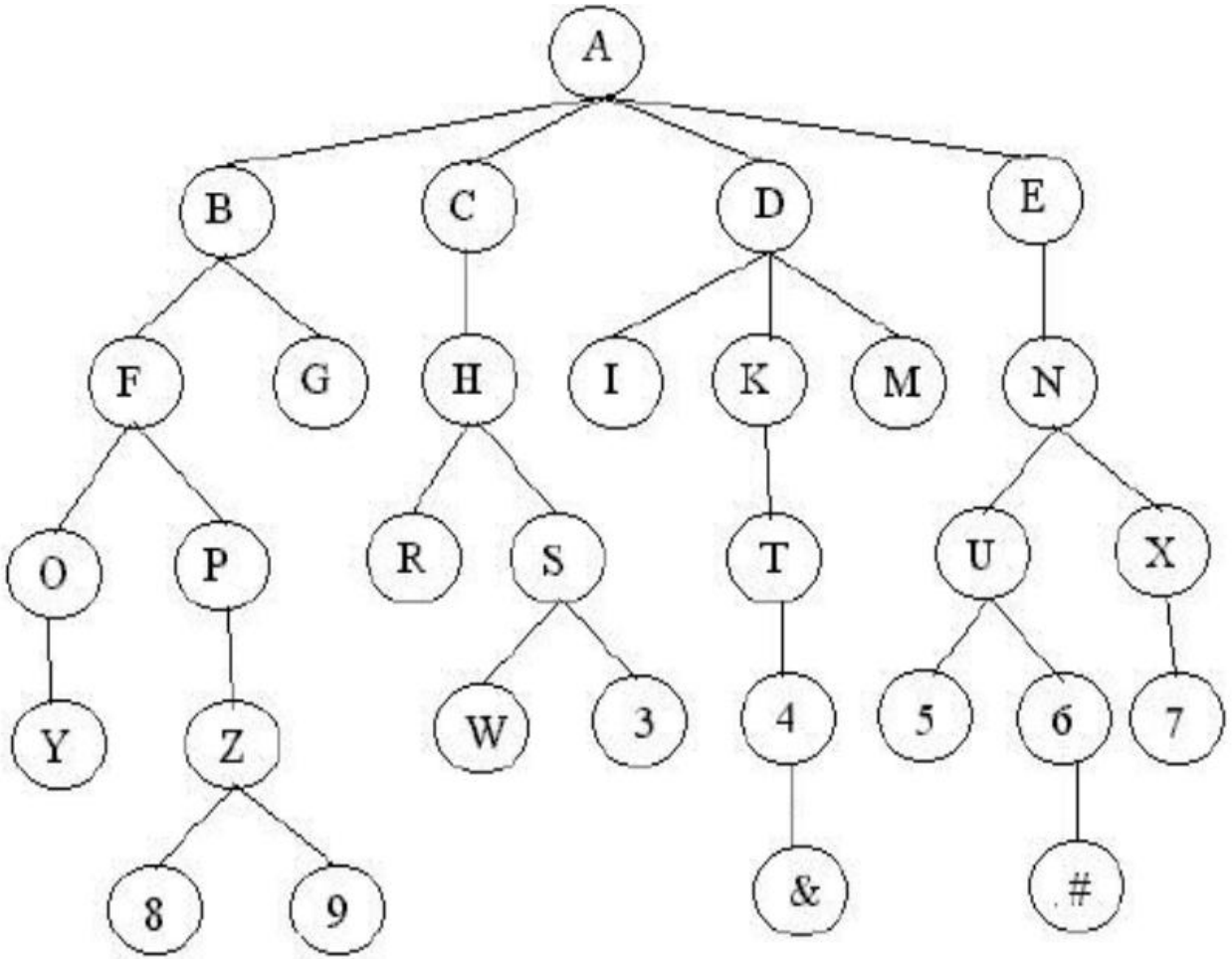
Convenciona-se que:

- A profundidade da raiz é 1.
- A Profundidade da árvore é 0.



Exercícios

Responda as questões tendo como base a árvore ao lado:



Exercícios

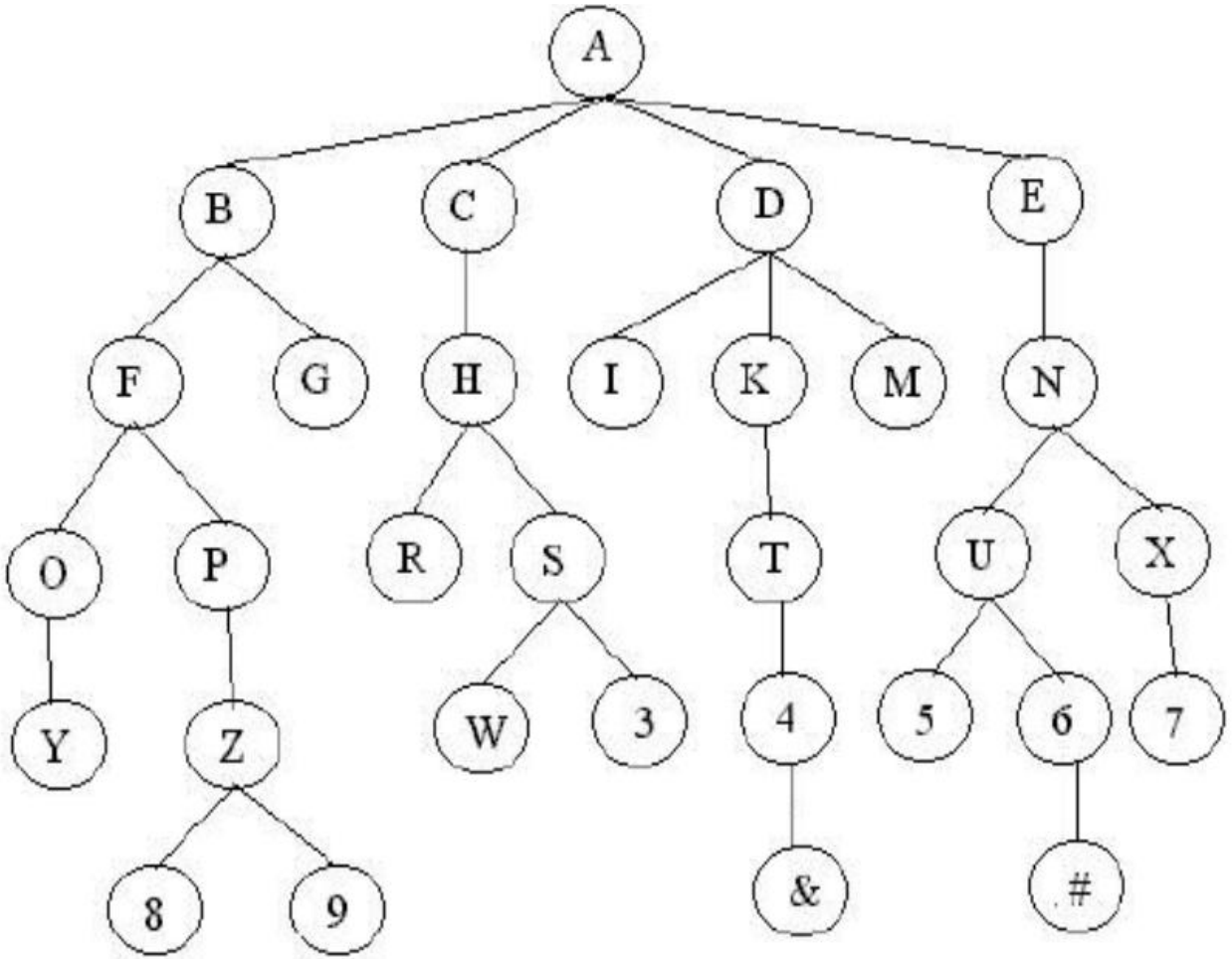
1. Qual é a raiz da árvore?

2. Quais são os nós terminais?

3. Qual o grau da árvore?

4. Qual o nível da árvore?

5. Quais são os nós descendentes do nó D?



Exercícios

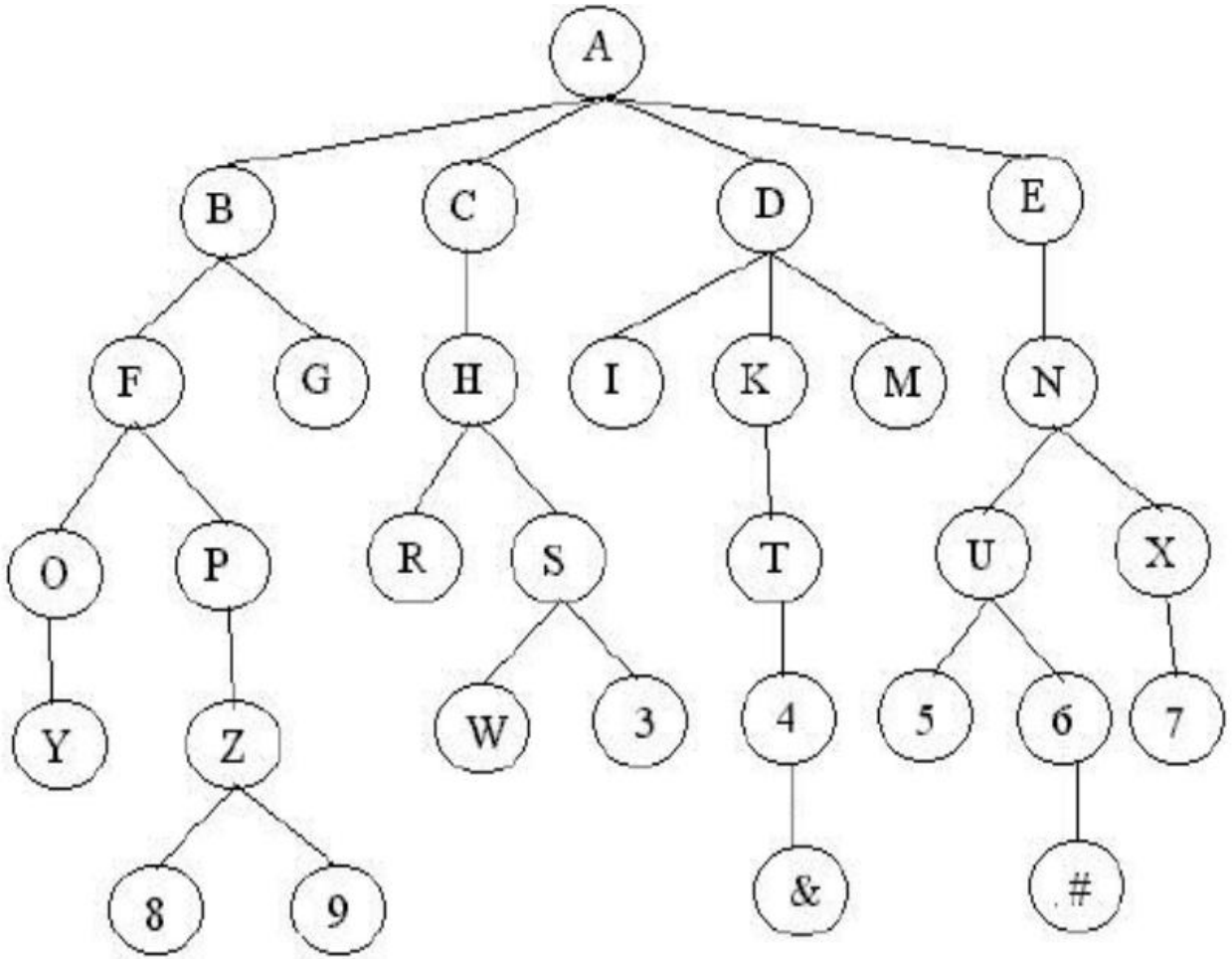
6. Quais são os nós ancestrais do nó #?

7. Os nós 4 e 5 são nós irmãos?

8. Há caminho entre os nós C e S?

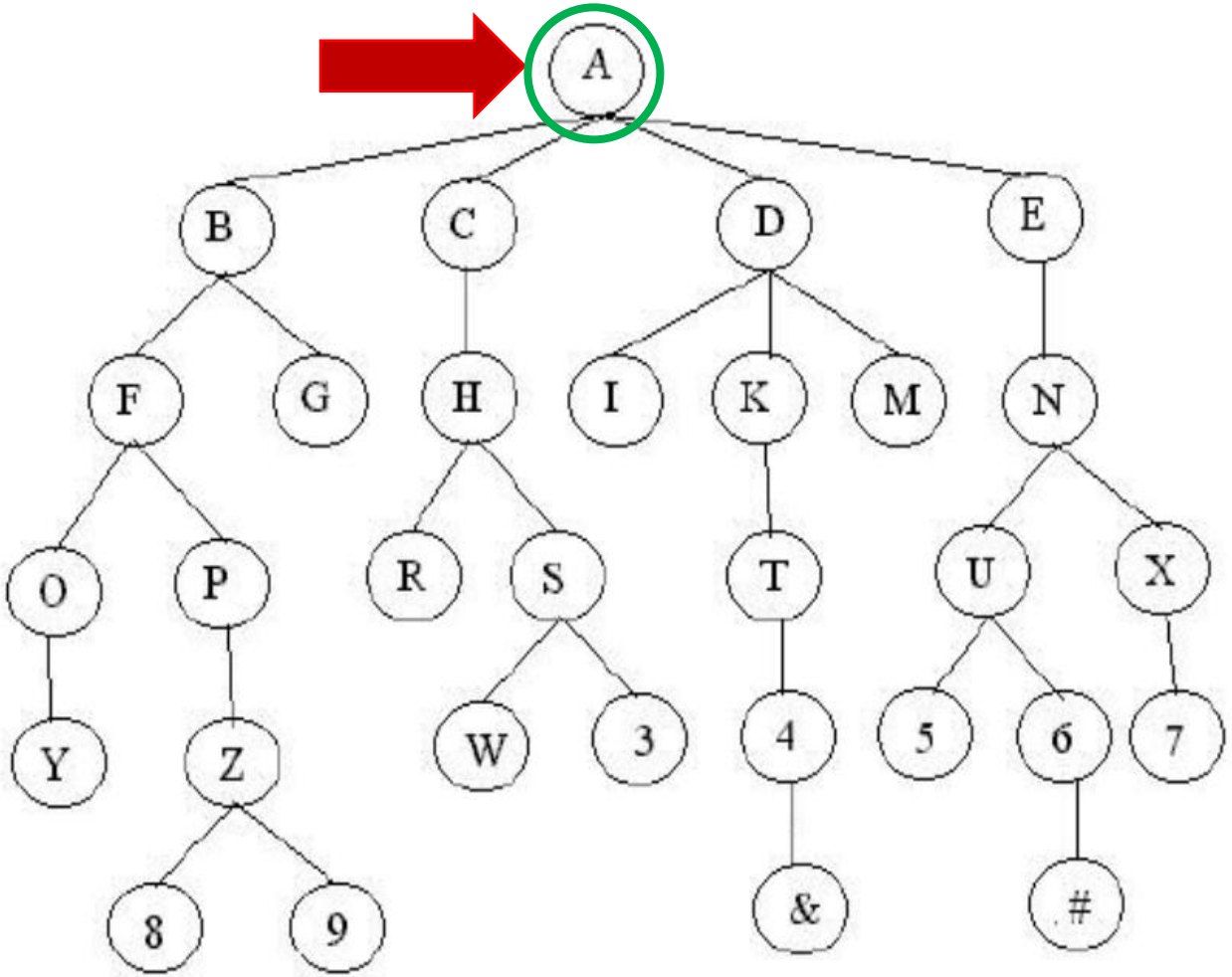
9. Qual o nível do nó 5?

10. Qual o grau do nó S?



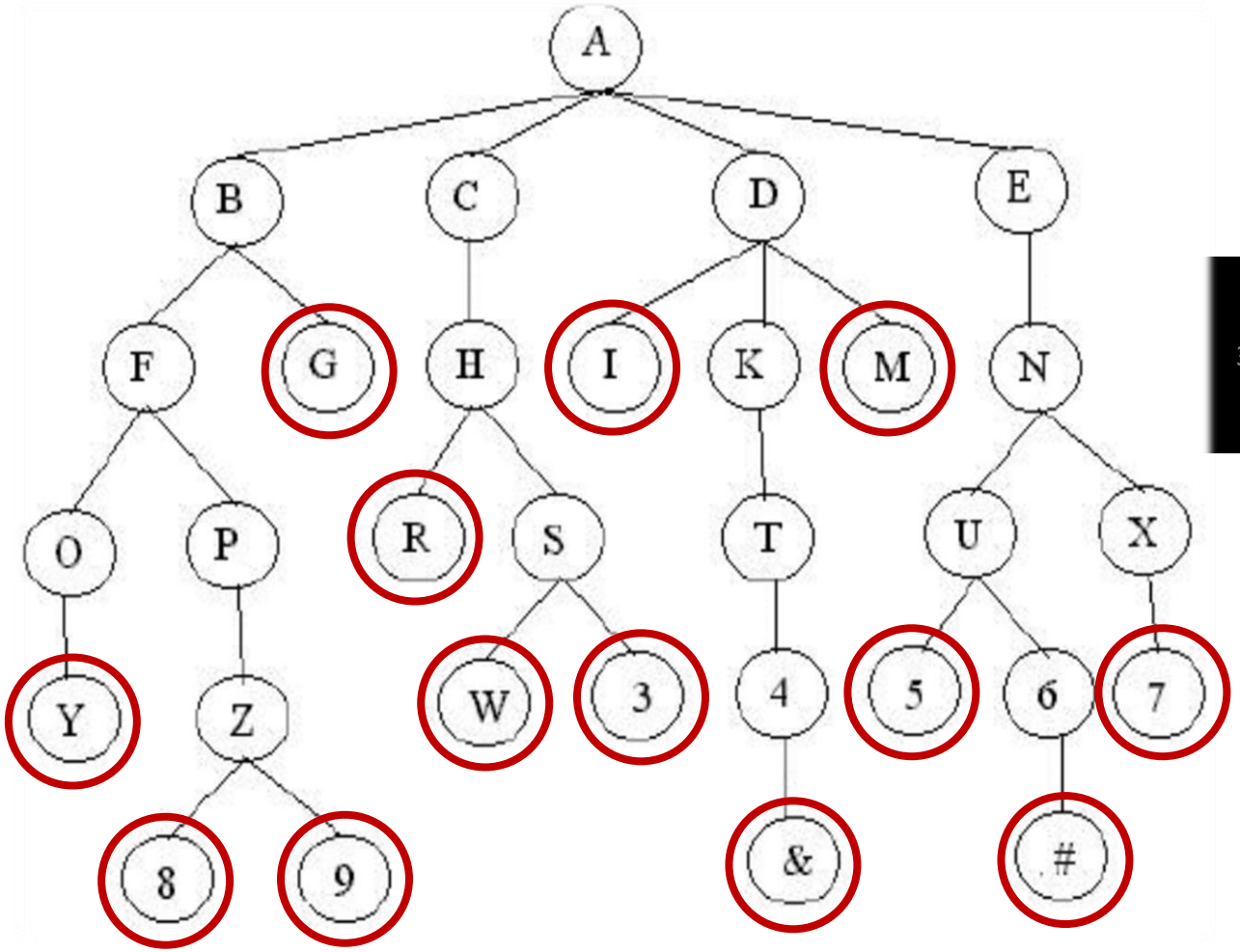
Exercícios

1. Qual é a raiz da árvore?



Exercícios

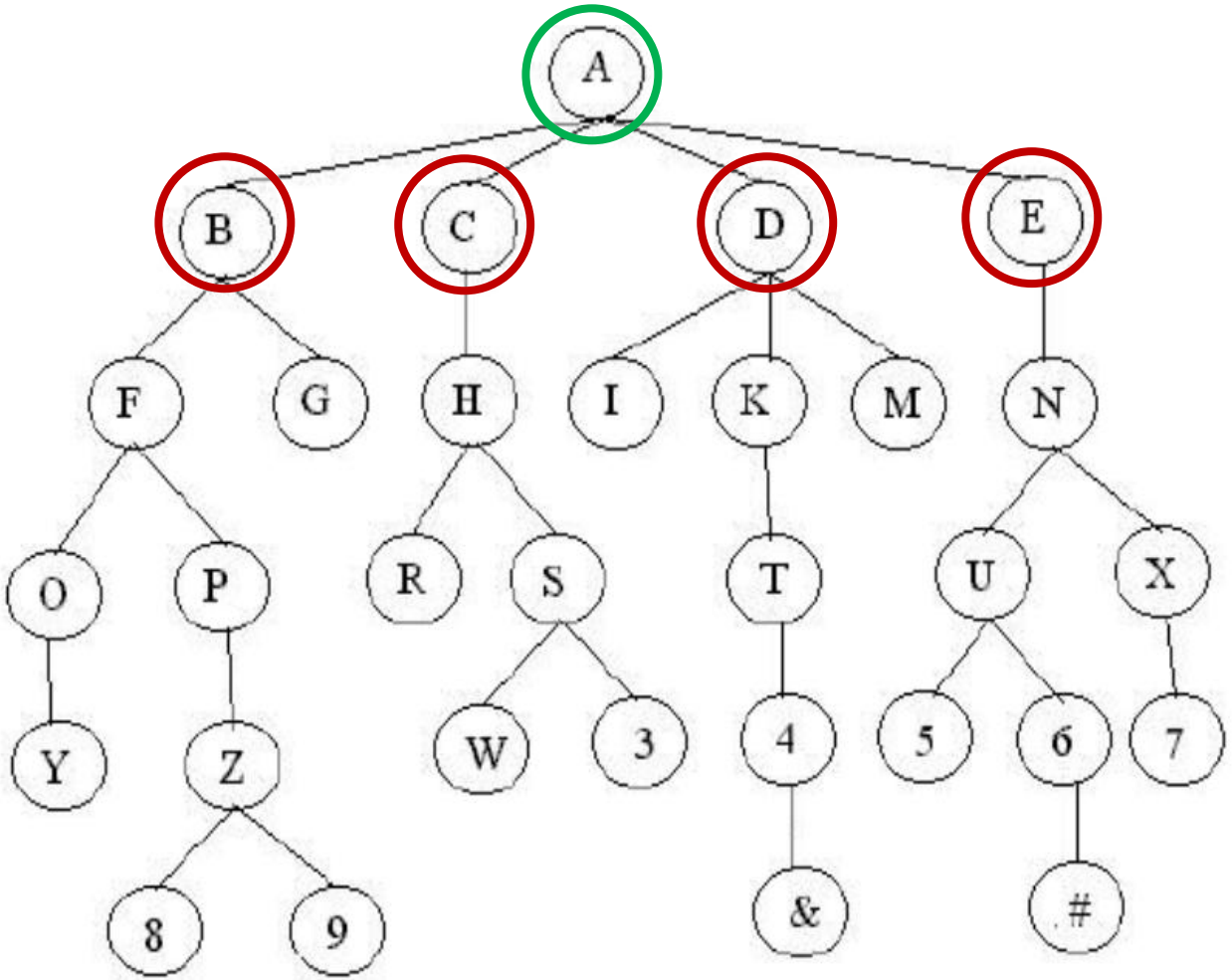
2. Quais são os nós terminais?



Exercícios

3. Qual o grau da árvore?

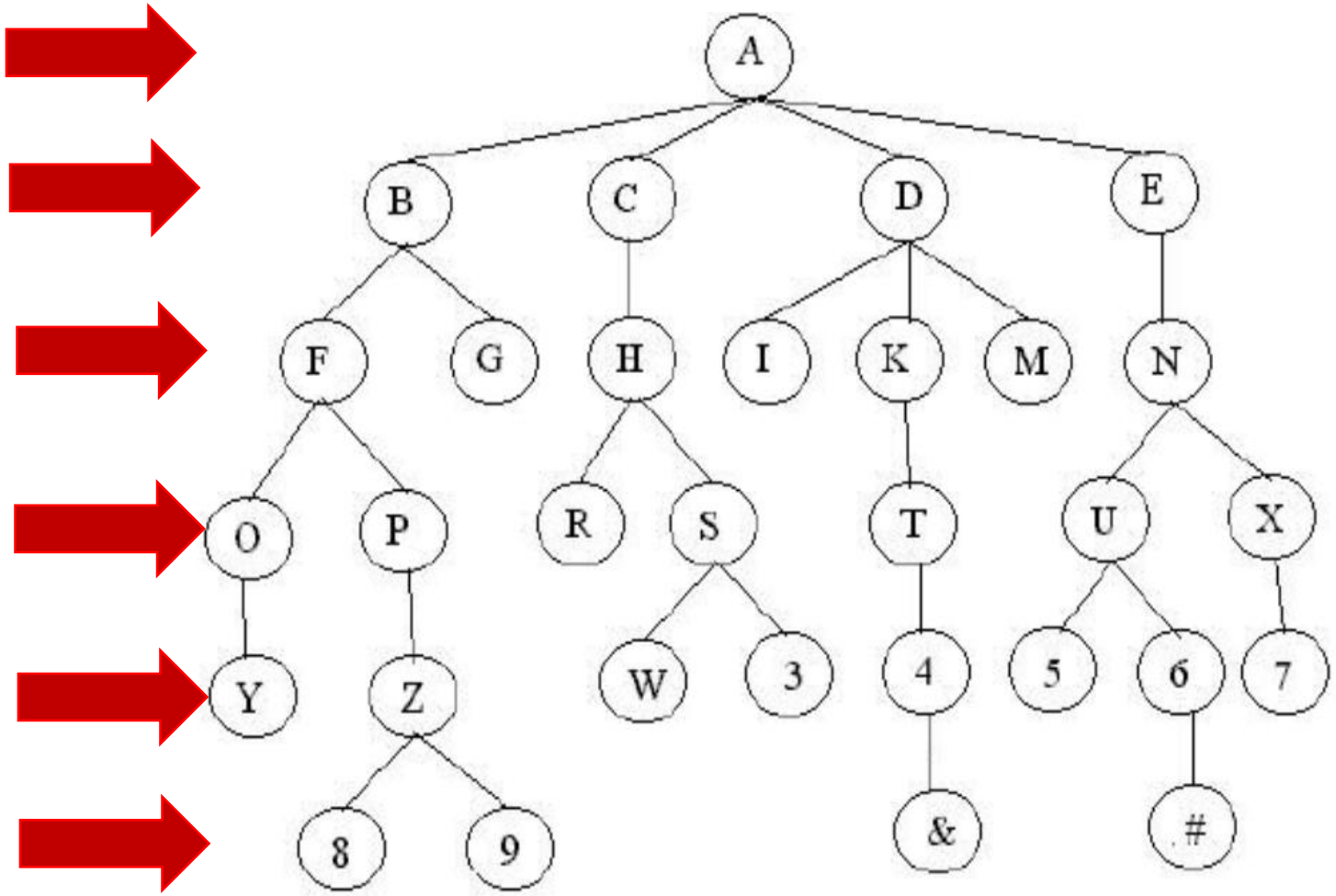
O nó A tem grau 4 porque tem 4 filhos.
É o nó que tem mais filhos.



Exercícios

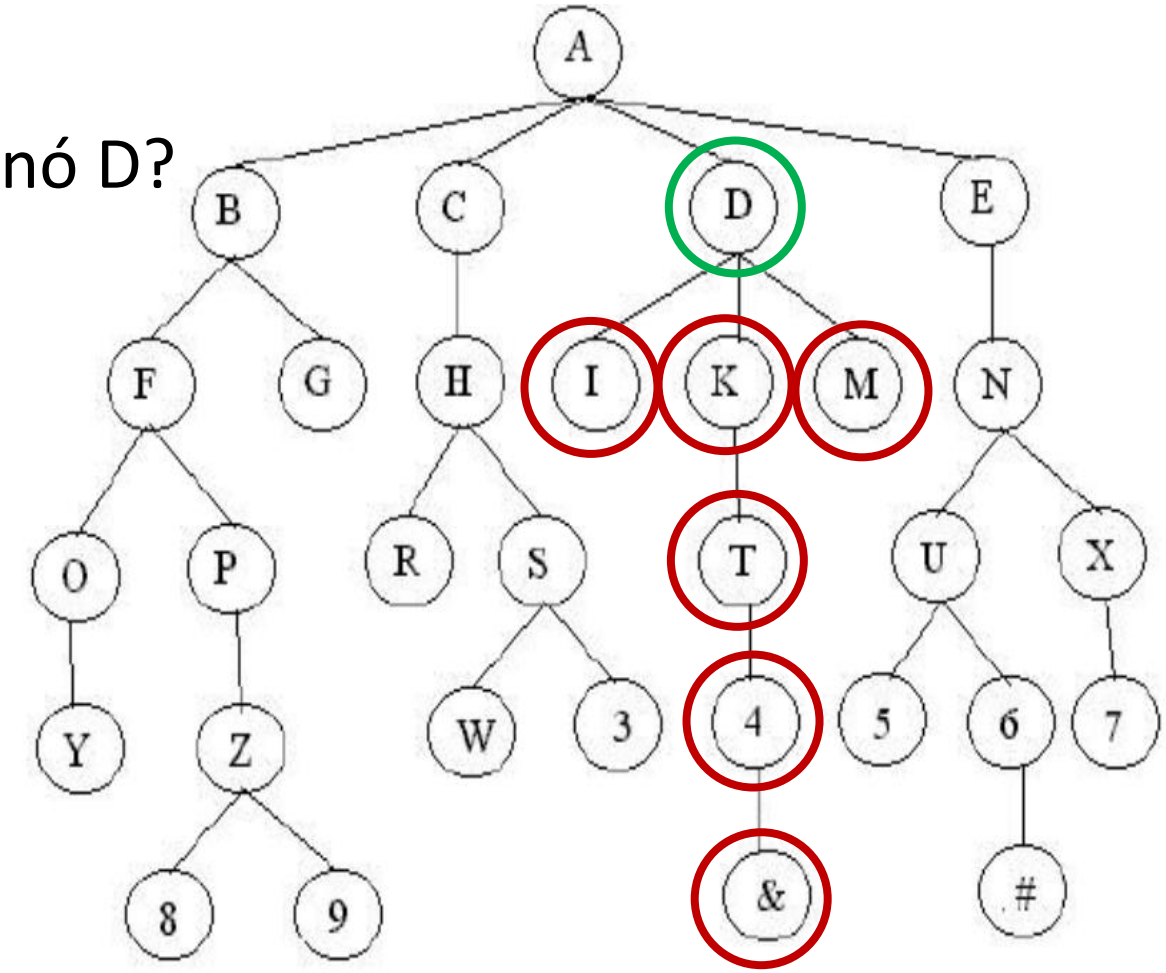
4. Qual o nível da árvore?

6 níveis



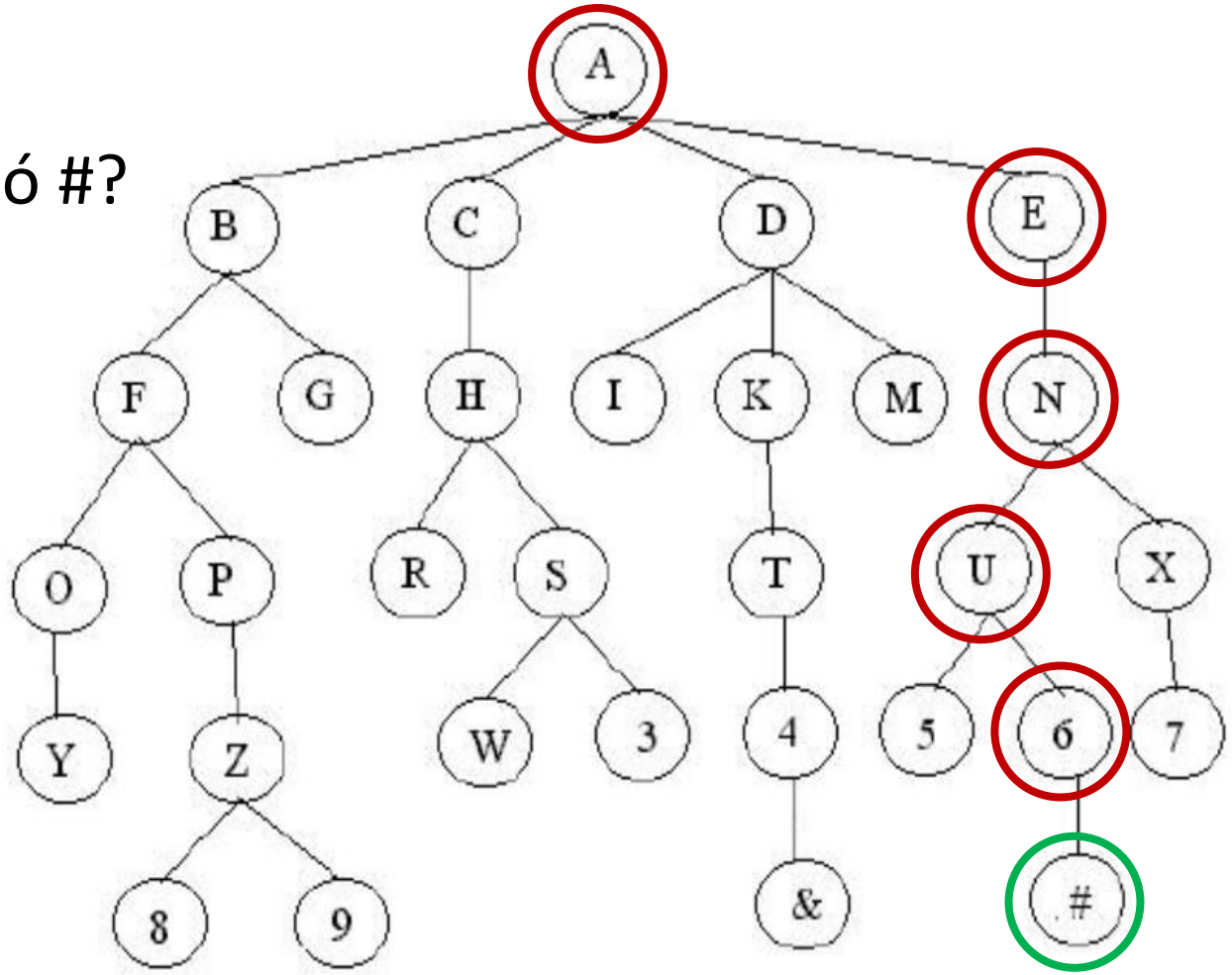
Exercícios

5. Quais são os nós descendentes do nó D?



Exercícios

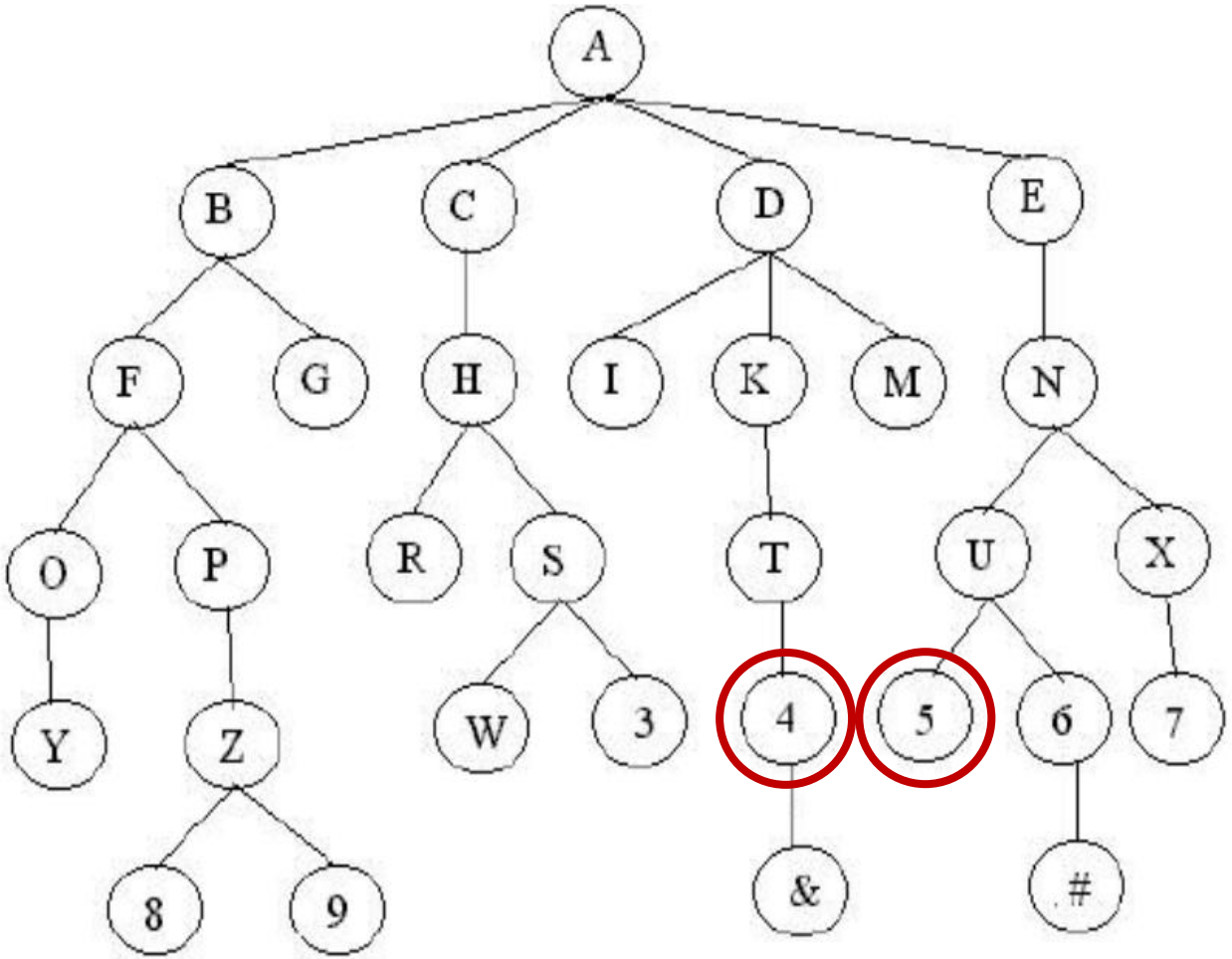
6. Quais são os nós ancestrais do nó #?



Exercícios

7. Os nós 4 e 5 são nós irmãos?

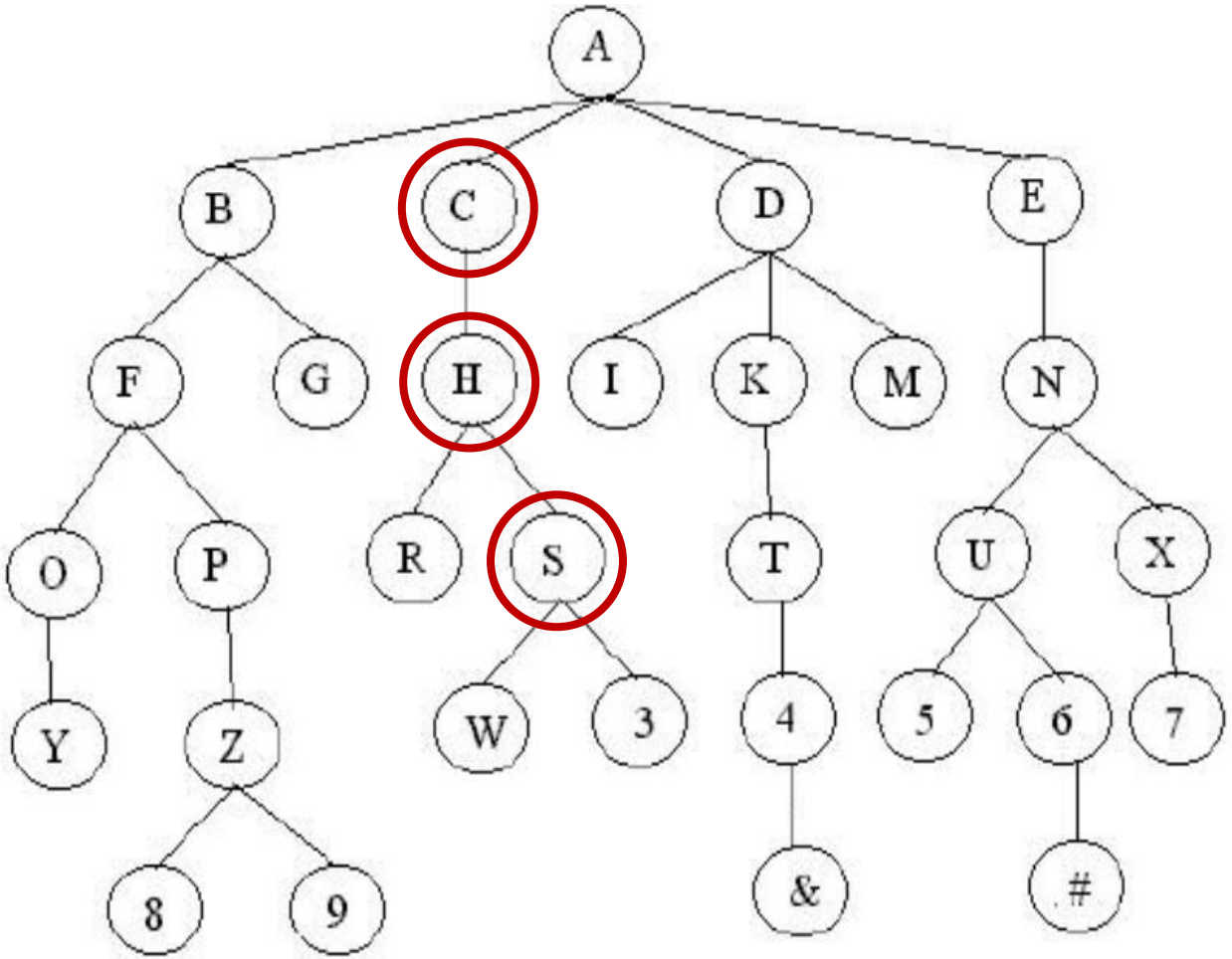
4 e 5 NÃO SÃO irmãos



Exercícios

8. Há caminho entre os nós C e S?

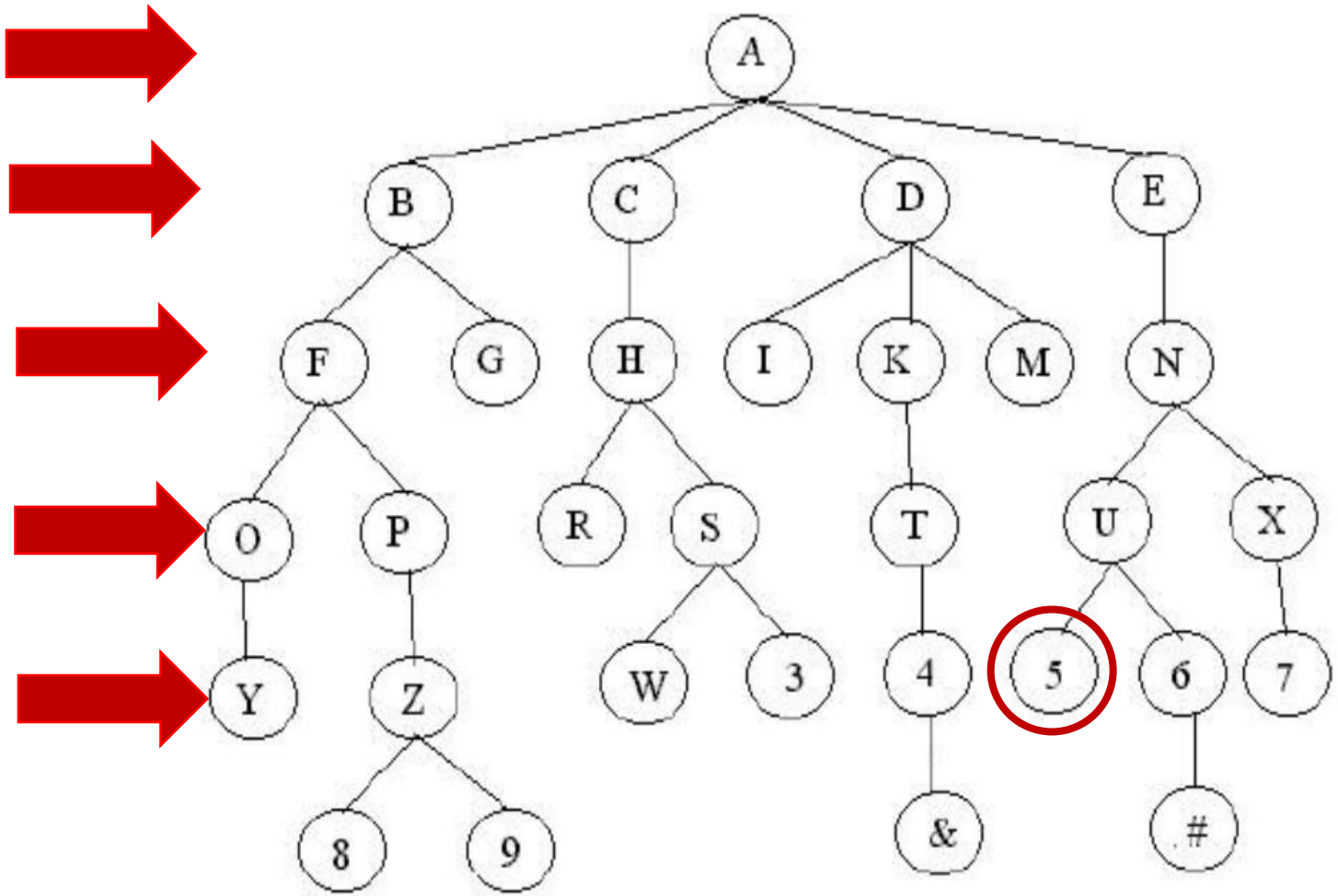
Sim, existe o caminho
(C, H, S)



Exercícios

9. Qual o nível do nó 5?

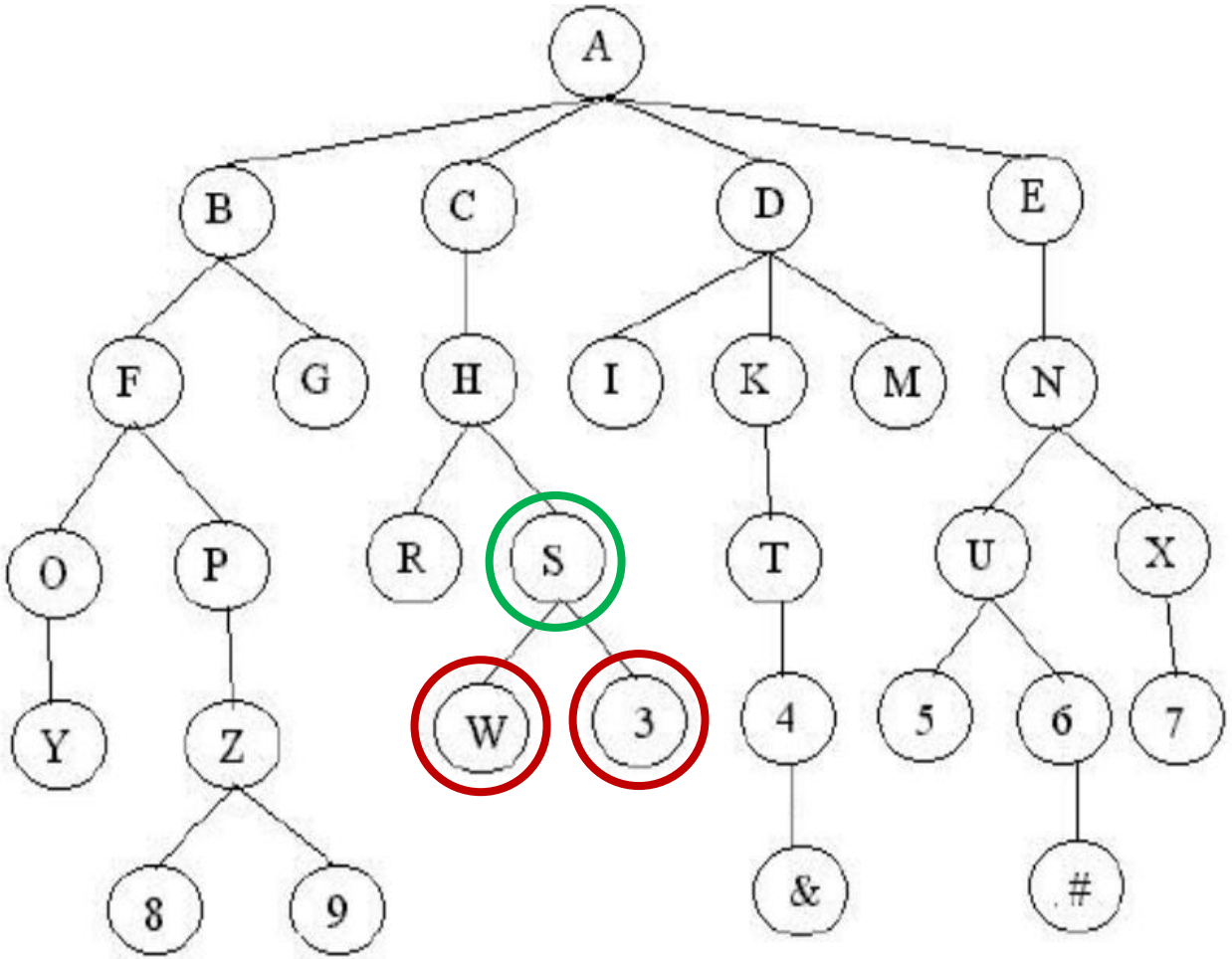
Nível 5



Exercícios

10. Qual o grau do nó S?

O nó S tem grau 2 porque tem 2 filhos.



Árvores

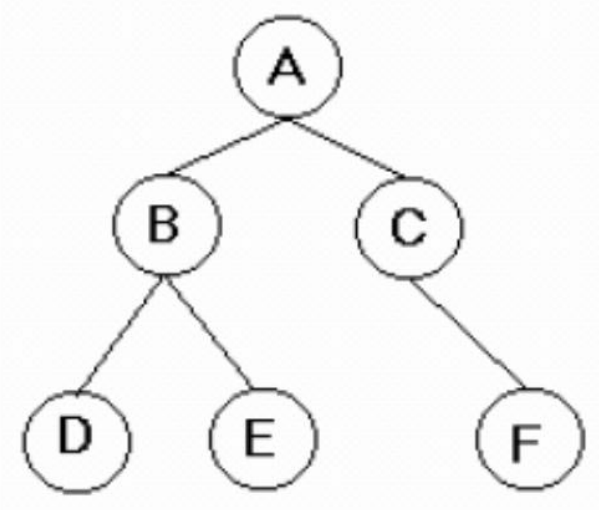
Existe uma ampla variedade de dados que são comumente organizados sob a forma de árvores hierárquicas utilizadas recorrentemente em nosso dia a dia.

Exemplos são a organização administrativa de empresas (Diretor, Gerentes, Chefes de Departamento, etc), a tabela de jogos de uma disputa esportiva (rodada classificatória, oitavas de final, quartas de final, etc) taxonomias (botânica, biologia, etc) dentre muitos outros.

Árvores

Toda árvore é composta por elementos chamados nós.

Tais nós são organizados de uma maneira hierárquica, ou seja, ligados logicamente uns aos outros de forma a mimetizar ramificações de uma árvore.



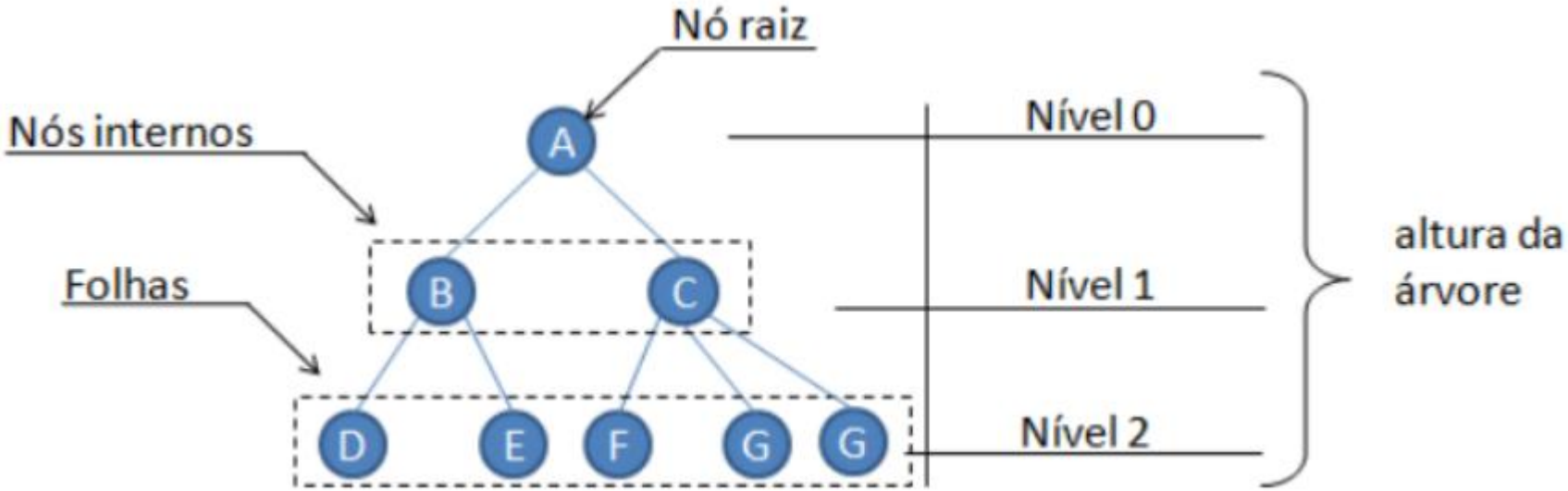
Árvores

Embora na natureza árvores ocorram naturalmente de forma que a raiz seja localizada na parte inferior (próxima ao chão) e seus galhos se ramifiquem para cima, em computação, árvores são representadas na ordem inversa, ou seja, raiz no topo e ramificações se expandindo para baixo.

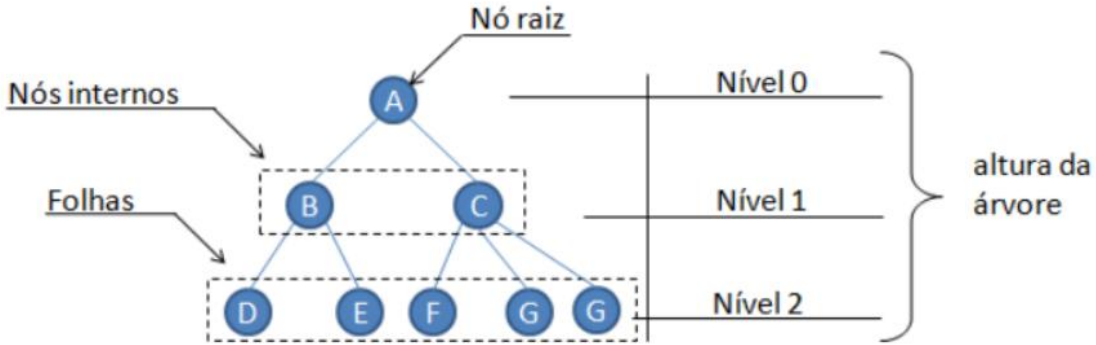
Árvores

O nó localizado mais acima é chamado de nó raiz.

Ele deve ser único para cada árvore servindo alternativamente como um cabeçalho da estrutura de dados.



Árvores



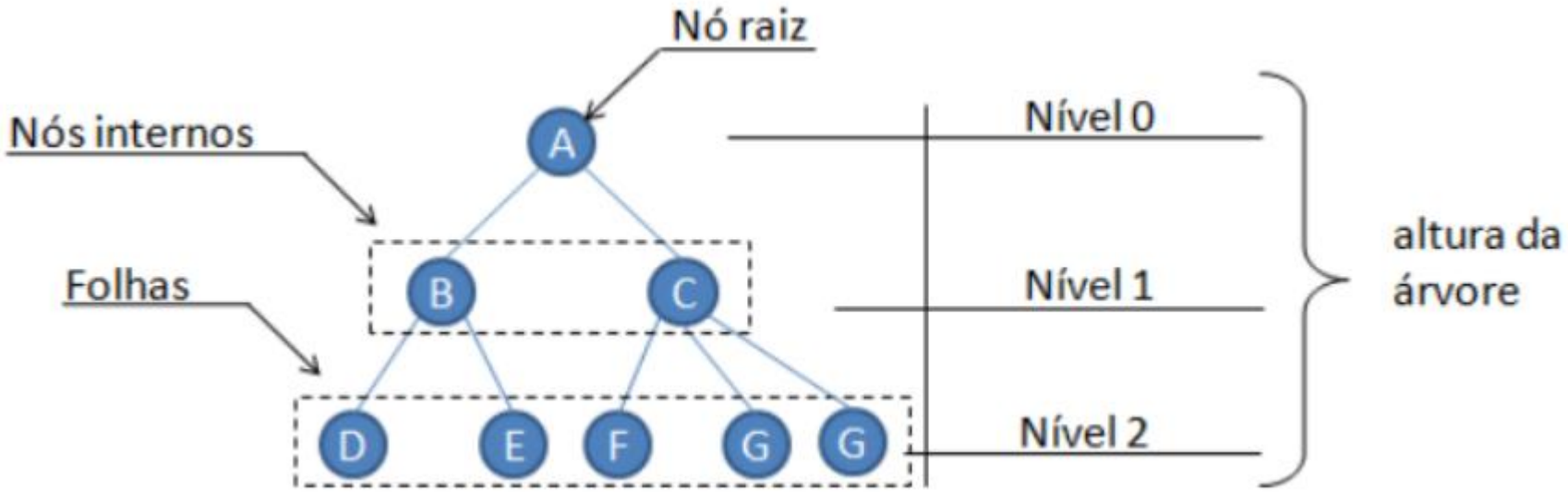
Outro fator que diferencia o nó raiz dos demais nós é o fato de que ele não possuir nós ancestrais.

Tal fato traz a nossa atenção outro conceito importante, o fato de que as relações entre nós em uma árvore são nomeadas.

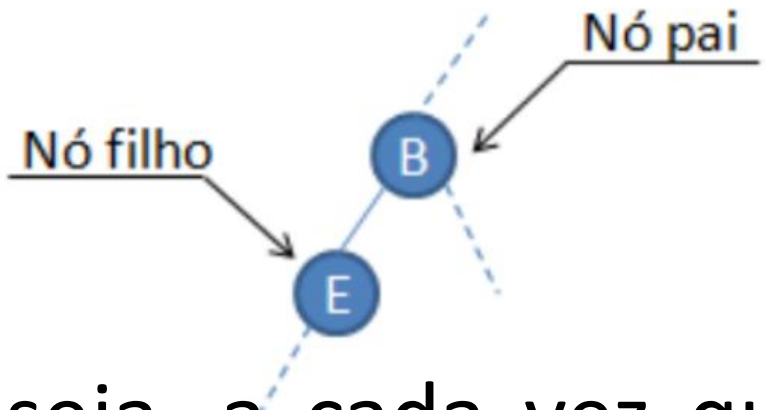
Um nó A é dito ancestral ou pai de outro nó B se B é expandido a partir de A. Ainda B é dito filho de A.

Árvores

Os nós que não possuem nenhum filho são chamados de nós folha ou nós terminais e todos os demais nós que não são nem o nó raiz nem folhas são chamados de nós internos.



Árvores



Cada nível de ancestralidade, ou seja, a cada vez que uma nova linha de expansão ocorre em uma árvore dizemos que um novo nível de ramificação é criado.

Os níveis são numerados a partir da raiz (nível 0) até o último nível de expansão (nível N) que conterá apenas nós folha. Por fim, o número total de níveis de uma árvore é chamado de altura da arvore.

Implementando Árvores Binárias Usando Arrays

Existem diversas formas de se implementar árvores binárias. Veremos a implementação via arrays estáticos e via estruturas dinâmicas ligadas.

A implementação via arrays requer que a árvore binária a ser codificada seja uma árvore binária completa ou quase completa. Tal fato impõe a restrição de que toda a estrutura da árvore deve ser conhecida antes de se criar a estrutura de dados para acomodá-la.

Implementando Árvores Binárias Usando Arrays

Para a criação do espaço de armazenamento basta que se aloque (estaticamente via declaração do tamanho do array no programa ou de maneira dinâmica via malloc).

A construção da árvore se dá via a inserção dos elementos da árvore no array seguindo o modelo de numeração de nós visto no exemplo de Grafos.

Implementando Árvores Binárias Usando Arrays

A implementação de árvores binárias via array serve para resolver problemas os quais todos os elementos da árvore são conhecidos a priori.

O ponto chave na solução de tais problemas recai na busca de itens dentro da estrutura da árvore, que em muitos casos pode ser consideravelmente mais rápido do que a busca linear ou sequencial.

Busca em Árvores Binárias

Não existe uma ordem “natural” para se percorrer uma árvore binária.

No entanto três formas básicas de busca são definidas:

- Ordem Anterior (percurso em profundidade)
- Ordem (ou em ordem simétrica)
- Ordem Posterior

Busca em Árvores Binárias

As formas de busca aqui descritas possuem implementação intuitiva via algoritmos recursivos.

Em ordem anterior, também chamada de busca PreOrdem ou busca em profundidade, os elementos da árvore são inspecionados a partir da raiz e da esquerda para a direita. direita Cada nó visitado tem seu valor inspecionado.

Busca em Árvores Binárias

A busca continua até que um nó folha da esquerda é encontrado. Neste momento a busca passa a acessar os elementos da direita.

```
ordemAnterior(no)
  IMPRIMA no.valor
  SE no.esq ≠ null ENTAO
    ordemAnterior(no esq)
  SE no.dir ≠ null ENTAO
    ordemAnterior(no.dir)
```

```
emOrdem(no)
  SE no.esq ≠ null ENTAO
    emOrdem(no.esq)
  IMPRIMA no.valor
  SE no.dir ≠ null ENTAO
    emOrdem(no.dir)
```

```
ordemPosterior(no)
  SE no.esq ≠ null ENTAO
    emOrdem(no.esq)
  SE no.dir ≠ null ENTAO
    emOrdem(no.dir)
  IMPRIMA no.valor
```

Busca em Árvores Binárias

Na busca emOrdem, os elementos da árvore são inspecionados a partir da raiz e da esquerda para a direita até que um nó folha seja encontrado (nó sem filhos).

A partir deste momento o nó é acessado a busca então analisa o no da direita.

O processo continua recursivamente percorrendo toda a árvore.

Busca em Árvores Binárias

Na busca em ordem posterior ou pós ordem a ideia é construir uma árvore de chamadas de função recursiva na pilha em que todos os elementos ele mentos da esquerda não percorridos seguidos dos elementos da direita.

A diferença reside no fato de que os elementos só serão acessados a partir do momento em que a recursão atingir a condição de parada.

Desta forma os elementos serão acessados na ordem inversa versa a qual eles foram percorridos.

Árvores Binárias Usando Nós Dinâmicos Ligados

Para implementar uma árvore binária baseada em nós dinâmicos faz-se faz se necessário:

- Definir uma estrutura de dados capaz de acomodar os dados a serem organizados sob a forma de uma árvore binária;
- Definir nesta estrutura ponteiros para as sub-árvores sub árvores da esquerda e da direita;
- Definir a interface ou lista de métodos que manipularão tal estrutura.

```
public class No {
    int dado;
    int level;
    No direita;
    No esquerda;
}
```

Árvores Binárias Usando Nós Dinâmicos Ligados

Árvore com link para o nó pai. Isso permite que não precisemos utilizar recursividade:

```
public class No {
    int dado;
    int level;
    No pai;
    No direita;
    No esquerda;
}
```

A estrutura acima define um nó para uma árvore binária. Note que com a inclusão do campo int level (nível) permite a identificação de em que nível da árvore o nó se encontra.

Árvores Binárias Usando Nós Dinâmicos Ligados

Métodos/funções para Árvores Binárias Dinâmicas

- inserir à direita
- inserir à esquerda
- retirar da direita
- retirar da esquerda
- juntar uma árvore à outra
- tamanho da árvore
- verifica se é folha
- retorna árvore da esquerda
- retorna árvore da direita
- outros...

Exemplo de Árvores

No.java × Arvore.java × Exemplo_Arvore.java ×

Source History

```java
package exemplo_arvore;

public class No {              // Classe do nó
    int info;                  // Informação tipo inteiro
    No left;                   // Filho à esquerda
    No right;                  // FIlho à direita

  public No(int info) {        // Método Construtor para Inicializa os atributos da classe
    this.info = info;          // Inicializa a informação com o parâmetro recebido
    this.left = null;          // Inicializa o filho à esquerda como vazio
    this.right = null;         // Inicializa o filho à direita como vazio
  }
}
```

Exe

No.java | Arvore.java | Exemplo_Arvore.java

Source | History

```java
package exemplo_arvore;

public class Arvore {
  No root = null;                                   // Nó pai, ou raíz

  public void insert(int info, No place) {          // algoritmo para inserir uma informação
    if (place == null) {                            // Se o local está vazio, então a árvore está vazia
      root = new No(info);                          // e o root vai receber o novo nó
      System.out.print(" " + info);                 // Imprime a informação inserida
    } else if (info < place.info) {                 // Senão, se a informação é menor do que a do local
      if (place.left == null) {                     // Se o filho da esquerda está vazio
        place.left = new No(info);                  // Então insere à esquerda o novo nó
        System.out.print(" " + info);               // Imprime a informação inserida
      } else {                                      // São, se não for vazio
        insert(info, place.left);                   // Pula para o filho à esquerda usando a recursividade
      }
    } else if (info > place.info) {                 // Senão, se a informação é maior que a do local
      if (place.right == null) {                    // Se o filho à direita está vazio
        place.right = new No(info);                 // Insere a informação em um novo nó à direita
        System.out.print(" " + info);               // e imprime a informação
      } else {                                      // Senão, se o nó à direita não está vazio
        insert(info, place.right);                  // Pula para o filho à direita usando a recursividade
      }
    }
  }
```

Exemplo de Árvores

No.java × | Arvore.java × | Exemplo_Arvore.java ×

Source | History

```
26
27  public void preOrder(No place) {            // Algoritmo de navegação em pré ordem
28      System.out.print(" " + place.info);     // Primeiro imprime a informação do nó
29      if (place.left != null) {               // Se o filho à esquerda não está vazio
30          preOrder(place.left);               // Pula para o filho à esquerda pela recursividade
31      }
32      if (place.right != null) {              // Se filho à direita não está vazio
33          preOrder(place.right);              // Pula para o filho à direita pela recursividade
34      }
35  }
```

Exemplo de Árvores

No.java × Arvore.java × Exemplo_Arvore.java ×

Source History

```
36
37  public void inOrder(No place) {              // Algoritmo de navegação em ordem
38      if (place.left != null) {                // Se o filho à esquerda não está vazio
39          inOrder(place.left);                 // Pula para o filho à esquerda pela recursividade
40      }
41      System.out.print(" " + place.info);      // Depois imprime a informação do nó
42      if (place.right != null) {               // Se filho à direita não está vazio
43          inOrder(place.right);                // Pula para o filho à direita pela recursividade
44      }
45  }
```

Exemplo de Árvores

No.java × Arvore.java × Exemplo_Arvore.java ×

Source History

```
46
47      public void postOrder(No place) {          // Algoritmo de navegação em pós ordem
48          if (place.left != null) {              // Se o filho à esquerda não está vazio
49              postOrder(place.left);             // Pula para o filho à esquerda pela recursividade
50          }
51          if (place.right != null) {             // Se filho à direita não está vazio
52              postOrder(place.right);            // Pula para o filho à direita pela recursividade
53          }
54          System.out.print(" " + place.info);    // Por último, imprime a informação do nó
55      }
56
57  }
58
```

No.java | Arvore.java | Exemplo_Arvore.java

Source | History

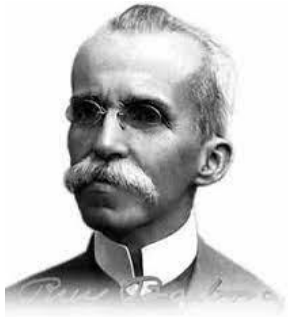
```java
package exemplo_arvore;

import java.util.Random;                                    // Usa a biblioteca de randomização

public class Exemplo_Arvore {

    public static void main(String[] args) {
        Random rand = new Random();                         // Inicializa a randomização
        Arvore arvore = new Arvore();                       // Cria uma árvore binária

            for (int i = 0; i < 10; i++) {                  // Fazer 10 vezes
                arvore.insert(rand.nextInt(100), arvore.root);  // Insere um elemento aleatório de 0 até 99
            }

            System.out.println("\nPRE-ORDER:");
            arvore.preOrder(arvore.root);                   // Navega a árvore em pré ordem

            System.out.println("\nIN-ORDER:");
            arvore.inOrder(arvore.root);                    // Navega a árvore em ordem

            System.out.println("\nPOST-ORDER:");
            arvore.postOrder(arvore.root);                  // Navega a árvore em pós ordem
    }
}
```

Exemplo de Árvores

Output - Exemplo_Arvore (run)

```
run:
 21 56 90 6 60 16 63 43 18 0
PRE-ORDER:
 21 6 0 16 18 56 43 90 60 63
IN-ORDER:
 0 6 16 18 21 43 56 60 63 90
POST-ORDER:
 0 18 16 6 43 63 60 90 56 21BUILD SUCCESSFUL (total time: 1 second)
```

"Maior que a tristeza de não haver vencido é a vergonha de não ter lutado"



Rui Barbosa

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br